



《程序语言理论与编译技术》 笔试备考册



编纂人：刘鼎琨

2025.4

笔试范围：Lecture1 ~ Lecture21

- Lecture 1: Introduction
- Lecture 2: Introduction of OCaml
- Lecture 3: Functional programming, recursions, and first-class functions
- Lecture 4: Alphabet, strings and languages
- Lecture 5: Regular languages, regular expressions, and automata
- Lecture 6: Context-free languages, grammars, and push-down automata
- Lecture 7: Designing a language
- Lecture 8: Lambda calculus
- Lecture 9: Simply types
- Lecture 10: Subtyping
- Lecture 11: Recursive types
- Lecture 12: Polymorphism
- Lecture 13: Lexing and parsing, recursive-descent parsing
- Lecture 14: Table-driven parsing: LL(k)
- Lecture 15: Table-driven parsing: LR(k)
- Lecture 16: Attribute grammars
- Lecture 17: Syntax-directed translation
- Lecture 18: Intermediate representations
- Lecture 19: Intermediate representation (cont)
- Lecture 20: Intermediate code generation
- Lecture 21: Intermediate code generation (cont)

说明：本文档包含的重点为往年真题、作业题，以及复习课重点，未包含全部课件考点。

机试范围：Lecture1 ~ Lecture21 对应的 Tutorial，以及机器代码生成的代码部分

- Tutorial 1: Basic programming tasks of OCaml
- Tutorial 2: Advanced programming tasks of OCaml
- Tutorial 3: Object-oriented programming in OCaml
- Tutorial 4: Writing a parser
- Tutorial 5: Constructing an interpreter
- Tutorial 6: Constructing a code generator

一、 OCaml 及函数式编程的概念

- (1) 往年题说明：由于本届第一次讲解 OCaml，所以没有体现。
- (2) 重点一：根据 OCaml 程序确定函数类型。复习课。
- (3) 重点二：判定 OCaml 程序的问题。复习课。
- (4) 重点三：函数式编程的重点概念。复习课。

二、 文法 Grammar

- (1) 往年题说明：是整张试卷的核心逻辑，考点集中于 LL(1)和 LR(0)上。
- (2) 重点一：语法树的构建。
区分清楚语法树、抽象语法树、附注语法树。作业 1.1
- (3) 重点二：设计无二义性语法。作业 1.2。复习课。
- (4) 重点三：根据文法确定含义，并用自然语言表达，复习课。

三、 自动机 FSA

- (1) 往年题说明：一般为第一题。
 - a) NFA 接受字符串过程：按顺序推导即可，注意除了最后一个状态外，不要画双圈。


```
graph LR; start((start)) -- epsilon --> 0((0)); 0 -- epsilon --> 1((1)); 1 -- a --> 2((2)); 2 -- epsilon --> 3((3)); 3 -- b --> 4((4)); 4 -- epsilon --> 5((5)); 5 -- a --> 6((6)); 6 -- epsilon --> 7((7)); 7 -- a --> 8((8)); 8 -- epsilon --> 9((9)); 9 -- b --> 10((10)); 10 -- b --> 11(((11)));
```
 - b) NFA→DFA：根据题干确定每个 DFA 状态包含哪些 NFA 状态；
 - c) 最小 DFA：先根据是否可接受划分两部分；再根据不同输入下同一集合是否落入其他集合内进一步细化。
 - d) 自然语言描述；
 - e) 正则表达式书写：建议先写正则再总结自然语言。
- (2) 重点一：和往年题相反，正则表达式→NFA→DFA。作业 1.4。复习课。
利用汤普孙构造法构建 NFA。三种方法！
- (3) 重点二：根据自然语言描述，写出正则表达式/文法。作业 1.3
- (4) 重点三：和“往年题”接受字符串相反，判定哪些串不能接收。复习课。
- (5) 重点四：NFA 的分裂逻辑，见整理 19 页，复习课。

四、 Lambda 表达式

- (1) 往年题说明：往年题没有体现。
- (2) 重点一：代入表达式求取值，复习课。
- (3) 重点二：设计 Lambda 函数实现相关功能，如作业 2.2

五、 类型系统

- (1) 往年题说明：往年题没有体现。
- (2) 重点一：lambda 表达式的类型推断，如作业 2.3 2.4
 - a) T-Var：处理环境；
 - b) T-Abs：处理函数定义的推导，多参数需要多次应用。
 - c) T-App：应用。
- (3) 重点二：子类型推导证明，如作业 3.1 3.2
S-RCDWIDTH, S-RCDDEPTH, S-RCDPERM, S-TRANS, S-REFL, S-ARROW.

六、 递归类型与多态

- (1) 往年题说明：往年题没有体现。
- (2) 重点一：递归类型设计，如作业 3.3
要确定函数类型，递归函数需要利用 fixed-point combiner.
- (3) 重点二：stream 递归类型的应用，如作业 3.4
- (4) 重点三：多态的主统一器（principal unifiers）求解，如作业 3.5
- (5) 重点四：church encoding 的案例分析，如作业 3.6

七、 Bottom-up 及 Top-down 编译流程计算

- (1) 往年题说明：是重点，一般位于二、三、四题
 - a) Top-down 的 LL(1)分析：
 - i. 语句的最左推导。
 - ii. 消除左递归：固定消除公式。
 - iii. First()和 Follow()求解：First()要按“产生式”和“文法”分开列举。
 - iv. 分析表，说明不是 LL(1)文法：行“非终结符”，列“终结符”。
 - v. 分析过程：分析栈 E、剩余串、操作三列。
 - b) Bottom-up 的 LR(0)、SLR(1)分析：
 - i. 项目集填写：要求填写核心项目以及非核心项目。
 - ii. 活前缀相关：执行左移、规约流程；有的题目让写出活前缀的正则表达式。
 - iii. SLR 分析表：利用 Follow 集，确定是规约还是移位。行“块号”、列“非终结符及终结符”。
 - iv. 分析过程：分析栈、状态栈、剩余串、操作四列。
 - c) 二义性问题、文法改进设计：
 - i. 构建两个不同的语法树即可。
 - ii. $E \rightarrow *E|\varepsilon$ 或者 $E \rightarrow *E|T$ ，优先级越高，越“靠下”。
- (2) 重点：和上述往年题类似。

八、 属性文法和语法制导规则

- (1) 往年题说明：一般为第五题，考察 SDD 而非 SDT。
 - a) 根据提供的属性进行列举，考虑全面。
 - b) 分析过程。
- (2) 重点一：考察属性文法，给定相关的函数和要求，写出属性，复习课。
- (3) 重点二：考察语法制导规则，类似“\$\$”的写法，可能需要画栈，复习课。

九、 常见中间表示 IR

- (1) 往年题说明：一般第六题，考察 Quadruple 编程。
- (2) 重点一：AST，CFG，画或者填空，复习课。
 - a) AST、语法树、附注语法树，三者需要区分。
 - b) Clang 指令；
- (3) 重点二：DAG，CFG，画或者填空，复习课。
 - Clang 指令；
- (4) 重点三：Quadruple，CFG，画或者填空，复习课。
- (5) 重点四：CFG，画或者填空，复习课。

十、 综合题目

- (1) 往年题说明：往年题考察对 C 语言指针等易错点的理解和分析，一般为第七题。
- (2) 难点：考察完整课程思路。

附录 A. LLVM IR 及 AST

Example 1: casting

```
int x = 1;
int doublenum (int para)
{
    para = para * 2;
    return para * 1;
}
int main()
{

    long a = 12;
    char b = 'b' , c;
    c = a + b;
    int result = doublenum(c);
    return result + x;
}
```

```
; ModuleID = './casting.cpp'
source_filename = "./casting.cpp"
target_datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target_triple = "x86_64-pc-linux-gnu"
```

```
@x = dso_local global i32 1, align 4
```

```
; Function Attrs: mustprogress noinline nounwind optnone uwtable
define dso_local noundef i32 @_Z9doublenumi(i32 noundef %0) #0 {
    %2 = alloca i32, align 4
    store i32 %0, i32* %2, align 4
    %3 = load i32, i32* %2, align 4
    %4 = mul nsw i32 %3, 2
    store i32 %4, i32* %2, align 4
    %5 = load i32, i32* %2, align 4
    %6 = mul nsw i32 %5, 1
    ret i32 %6
}
```

```
; Function Attrs: mustprogress noinline norecurse nounwind optnone uwtable
define dso_local noundef i32 @main() #1 {
```

```

%1 = alloca i32, align 4
%2 = alloca i64, align 8
%3 = alloca i8, align 1
%4 = alloca i8, align 1
%5 = alloca i32, align 4
store i32 0, i32* %1, align 4
store i64 12, i64* %2, align 8
store i8 98, i8* %3, align 1
%6 = load i64, i64* %2, align 8
%7 = load i8, i8* %3, align 1
%8 = sext i8 %7 to i64
%9 = add nsw i64 %6, %8
%10 = trunc i64 %9 to i8
store i8 %10, i8* %4, align 1
%11 = load i8, i8* %4, align 1
%12 = sext i8 %11 to i32
%13 = call noundef i32 @_Z9doublenumi(i32 noundef %12)
store i32 %13, i32* %5, align 4
%14 = load i32, i32* %5, align 4
%15 = load i32, i32* @x, align 4
%16 = add nsw i32 %14, %15
ret i32 %16
}

```

```

attributes #0 = { mustprogress noinline nounwind optnone uwtable "frame-
pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { mustprogress noinline norecurse nounwind optnone uwtable "frame-
pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

```

```

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

```

```

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 7, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 1}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{"Ubuntu clang version 14.0.6"}

```

```

`-FunctionDecl 0x1a85a90 <line:7:1, line:15:1> line:7:5 main 'int ()'
  |-CompoundStmt 0x1a860c0 <line:8:1, line:15:1>
    |-DeclStmt 0x1a85c10 <line:10:2, col:13>
      | `'-VarDecl 0x1a85b70 <col:2, col:11> col:7 used a 'long' cinit
      |   `'-ImplicitCastExpr 0x1a85bf8 <col:11> 'long' <IntegralCast>
      |     `'-IntegerLiteral 0x1a85bd8 <col:11> 'int' 12
    |-DeclStmt 0x1a85d48 <line:11:2, col:18>
      | |-VarDecl 0x1a85c38 <col:2, col:11> col:7 used b 'char' cinit
      | | `'-CharacterLiteral 0x1a85ca0 <col:11> 'char' 98
      | | `'-VarDecl 0x1a85cc8 <col:2, col:17> col:17 used c 'char'
    |-BinaryOperator 0x1a85e40 <line:12:2, col:10> 'char' lvalue '='
      | |-DeclRefExpr 0x1a85d60 <col:2> 'char' lvalue Var 0x1a85cc8 'c' 'char'
      | | `'-ImplicitCastExpr 0x1a85e28 <col:6, col:10> 'char' <IntegralCast>
      | |   `'-BinaryOperator 0x1a85e08 <col:6, col:10> 'long' '+'
      | |     |-ImplicitCastExpr 0x1a85dc0 <col:6> 'long' <LValueToRValue>
      | |     | `'-DeclRefExpr 0x1a85d80 <col:6> 'long' lvalue Var 0x1a85b70 'a' 'long'
      | |     | `'-ImplicitCastExpr 0x1a85df0 <col:10> 'long' <IntegralCast>
      | |     |   `'-ImplicitCastExpr 0x1a85dd8 <col:10> 'char' <LValueToRValue>
      | |     |     `'-DeclRefExpr 0x1a85da0 <col:10> 'char' lvalue Var 0x1a85c38 'b' 'char'
    |-DeclStmt 0x1a86008 <line:13:2, col:27>
      | `'-VarDecl 0x1a85e78 <col:2, col:26> col:6 used result 'int' cinit
      |   `'-CallExpr 0x1a85fb0 <col:15, col:26> 'int'
      |     |-ImplicitCastExpr 0x1a85f98 <col:15> 'int (*) (int)' <FunctionToPointerDecay>
      |     | `'-DeclRefExpr 0x1a85f48 <col:15> 'int (int)' lvalue Function 0x1a85830 'doublenum' 'int (int)'
      |     | `'-ImplicitCastExpr 0x1a85ff0 <col:25> 'int' <IntegralCast>
      |     |   `'-ImplicitCastExpr 0x1a85fd8 <col:25> 'char' <LValueToRValue>
      |     |     `'-DeclRefExpr 0x1a85f28 <col:25> 'char' lvalue Var 0x1a85cc8 'c' 'char'
    `'-ReturnStmt 0x1a860b0 <line:14:2, col:18>
      `'-BinaryOperator 0x1a86090 <col:9, col:18> 'int' '+'
        |-ImplicitCastExpr 0x1a86060 <col:9> 'int' <LValueToRValue>
        | `'-DeclRefExpr 0x1a86020 <col:9> 'int' lvalue Var 0x1a85e78 'result' 'int'
        `'-ImplicitCastExpr 0x1a86078 <col:18> 'int' <LValueToRValue>
          `'-DeclRefExpr 0x1a86040 <col:18> 'int' lvalue Var 0x1a85628 'x' 'int'

```


Example 2: loop

```
#include <iostream>

int main()
{
    int sum1 = 1;
    for (int i = 2; i < 10; i++)
    {
        sum1 += i;
    }

    int sum2 = 3, i = 4;
    while(i < 10)
    {
        sum2 += i;
        i++;
    }

    if(sum1 + sum2 < 100)
        std::cout << "less" << std::endl;
    else
        std::cout << "high" << std::endl;

    return sum1;
}
```

```
; ModuleID = './loop.cpp'
source_filename = "./loop.cpp"
target_datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-n8:16:32:64-S128"
target_triple = "x86_64-pc-linux-gnu"
```

```
module asm ".globl _ZSt21ios_base_library_initv"
```

```
%"class.std::basic_ostream" = type { i32 (...)**, %"class.std::basic_ios" }
%"class.std::basic_ios" = type { %"class.std::ios_base", %"class.std::basic_ostream"*,
i8,
i8, %"class.std::basic_streambuf"*, %"class.std::ctype"*, %"class.std::num_put"*, %"
class.std::num_get"* }
%"class.std::ios_base" = type { i32 (...)**, i64, i64, i32, i32,
i32, %"struct.std::ios_base::_Callback_list"*, %"struct.std::ios_base::_Words", [8
```

```

x                                     %"struct.std::ios_base::_Words"],
i32, %"struct.std::ios_base::_Words"*, %"class.std::locale" }
%"struct.std::ios_base::_Callback_list" = type
{ %"struct.std::ios_base::_Callback_list"*, void (i32, %"class.std::ios_base"*, i32)*,
i32, i32 }
%"struct.std::ios_base::_Words" = type { i8*, i64 }
%"class.std::locale" = type { %"class.std::locale::_Impl"* }
%"class.std::locale::_Impl" = type { i32, %"class.std::locale::facet"*,
i64, %"class.std::locale::facet"*, i8** }
%"class.std::locale::facet" = type <{ i32 (...)**, i32, [4 x i8] }>
%"class.std::basic_streambuf" = type { i32 (...)**, i8*, i8*, i8*, i8*, i8*,
i8*, %"class.std::locale" }
%"class.std::ctype" = type <{ %"class.std::locale::facet.base", [4 x
i8], %struct.__locale_struct*, i8, [7 x i8], i32*, i32*, i16*, i8, [256 x i8], [256 x i8], i8,
[6 x i8] }>
%"class.std::locale::facet.base" = type <{ i32 (...)**, i32 }>
%struct.__locale_struct = type { [13 x %struct.__locale_data*], i16*, i32*, i32*, [13 x
i8*] }
%struct.__locale_data = type opaque
%"class.std::num_put" = type { %"class.std::locale::facet.base", [4 x i8] }
%"class.std::num_get" = type { %"class.std::locale::facet.base", [4 x i8] }

```

```

@_ZSt4cout = external global %"class.std::basic_ostream", align 8
@.str = private unnamed_addr constant [5 x i8] c"less\00", align 1
@.str.1 = private unnamed_addr constant [5 x i8] c"high\00", align 1

```

```

; Function Attrs: mustprogress noline norecurse optnone uwtable

```

```

define dso_local noundef i32 @main() #0 {

```

```

    %1 = alloca i32, align 4
    %2 = alloca i32, align 4
    %3 = alloca i32, align 4
    %4 = alloca i32, align 4
    %5 = alloca i32, align 4
    store i32 0, i32* %1, align 4
    store i32 1, i32* %2, align 4
    store i32 2, i32* %3, align 4
    br label %6

```

```

6:                                     ; preds = %13, %0

```

```

    %7 = load i32, i32* %3, align 4
    %8 = icmp slt i32 %7, 10
    br i1 %8, label %9, label %16

```

```

9:                                     ; preds = %6

```

```
%10 = load i32, i32* %3, align 4
%11 = load i32, i32* %2, align 4
%12 = add nsw i32 %11, %10
store i32 %12, i32* %2, align 4
br label %13
```

```
13:                                     ; preds = %9
```

```
%14 = load i32, i32* %3, align 4
%15 = add nsw i32 %14, 1
store i32 %15, i32* %3, align 4
br label %6, !llvm.loop !6
```

```
16:                                     ; preds = %6
```

```
store i32 3, i32* %4, align 4
store i32 4, i32* %5, align 4
br label %17
```

```
17:                                     ; preds = %20, %16
```

```
%18 = load i32, i32* %5, align 4
%19 = icmp slt i32 %18, 10
br i1 %19, label %20, label %26
```

```
20:                                     ; preds = %17
```

```
%21 = load i32, i32* %5, align 4
%22 = load i32, i32* %4, align 4
%23 = add nsw i32 %22, %21
store i32 %23, i32* %4, align 4
%24 = load i32, i32* %5, align 4
%25 = add nsw i32 %24, 1
store i32 %25, i32* %5, align 4
br label %17, !llvm.loop !8
```

```
26:                                     ; preds = %17
```

```
%27 = load i32, i32* %2, align 4
%28 = load i32, i32* %4, align 4
%29 = add nsw i32 %27, %28
%30 = icmp slt i32 %29, 100
br i1 %30, label %31, label %34
```

```
31:                                     ; preds = %26
```

```
%32 = call @noundef_nonnull_align_8_dereferenceable(8) @"class.std::basic_ostream"*,
@_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc(%"class.std::basic_ostream"*,
noundef_nonnull_align_8_dereferenceable(8) @_ZSt4cout, i8* noundef
getelementptr inbounds ([5 x i8], [5 x i8]* @.str, i64 0, i64 0))
```

```

    %33 = call noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"*
    @_ZNSolsEPFRSoS_E(%"class.std::basic_ostream"* noundef nonnull align 8
    dereferenceable(8) %32, %"class.std::basic_ostream"*
    (%"class.std::basic_ostream"*)* noundef
    @_ZSt4endlcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_)
    br label %37

```

```

34:                                     ; preds = %26
    %35 = call noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"*
    @_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc(%"class.std::basic_o
    stream"* noundef nonnull align 8 dereferenceable(8) @_ZSt4cout, i8* noundef
    getelementptr inbounds ([5 x i8], [5 x i8]* @.str.1, i64 0, i64 0))
    %36 = call noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"*
    @_ZNSolsEPFRSoS_E(%"class.std::basic_ostream"* noundef nonnull align 8
    dereferenceable(8) %35, %"class.std::basic_ostream"*
    (%"class.std::basic_ostream"*)* noundef
    @_ZSt4endlcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_)
    br label %37

```

```

37:                                     ; preds = %34, %31
    %38 = load i32, i32* %2, align 4
    ret i32 %38
}

```

```

declare noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"*
@_ZStlsISt11char_traitsIcEERSt13basic_ostreamIcT_ES5_PKc(%"class.std::basic_o
stream"* noundef nonnull align 8 dereferenceable(8), i8* noundef) #1

```

```

declare noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"*
@_ZNSolsEPFRSoS_E(%"class.std::basic_ostream"* noundef nonnull align 8
dereferenceable(8), %"class.std::basic_ostream"* (%"class.std::basic_ostream"*)*
noundef) #1

```

```

declare noundef nonnull align 8 dereferenceable(8) @"class.std::basic_ostream"*
@_ZSt4endlcSt11char_traitsIcEERSt13basic_ostreamIT_T0_ES6_(%"class.std::basi
c_ostream"* noundef nonnull align 8 dereferenceable(8)) #1

```

```

attributes #0 = { mustprogress noline norecurse optnone uwtable "frame-
pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }
attributes #1 = { "frame-pointer"="all" "no-trapping-math"="true" "stack-protector-
buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

```

```
!llvm.module.flags = !{!0, !1, !2, !3, !4}  
!llvm.ident = !{!5}
```

```
!0 = !{i32 1, !"wchar_size", i32 4}  
!1 = !{i32 7, !"PIC Level", i32 2}  
!2 = !{i32 7, !"PIE Level", i32 2}  
!3 = !{i32 7, !"uwtable", i32 1}  
!4 = !{i32 7, !"frame-pointer", i32 2}  
!5 = !{"Ubuntu clang version 14.0.6"}  
!6 = distinct !{!6, !7}  
!7 = !{"!llvm.loop.mustprogress"}  
!8 = distinct !{!8, !7}
```

Example 3: loop2

```
int main()  
{  
    int sum1 = 1;  
    for (int i = 2; i < 10; i++)  
    {  
        sum1 += i;  
    }  
  
    int sum2 = 3, i = 4;  
    while(i < 10)  
    {  
        sum2 += i;  
        i++;  
    }  
  
    if(sum1 + sum2 < 100)  
        return sum1;  
    else  
        return sum2;  
}
```

```

`-FunctionDecl 0x23b2660 <loop_2.cpp:1:1, line:20:1> line:1:5 main 'int ()'
  |-CompoundStmt 0x23b2ee0 <line:2:1, line:20:1>
    |-DeclStmt 0x23b2810 <line:3:2, col:14>
      | `VarDecl 0x23b2788 <col:2, col:13> col:6 used sum1 'int' cinit
      |   `IntegerLiteral 0x23b27f0 <col:13> 'int' 1
    |-ForStmt 0x23b2a30 <line:4:2, line:7:2>
      |-DeclStmt 0x23b28c8 <line:4:7, col:16>
        | `VarDecl 0x23b2840 <col:7, col:15> col:11 used i 'int' cinit
        |   `IntegerLiteral 0x23b28a8 <col:15> 'int' 2
      |<<<NULL>>>
      |-BinaryOperator 0x23b2938 <col:18, col:22> 'bool' '<'
        |||-ImplicitCastExpr 0x23b2920 <col:18> 'int' <LValueToRValue>
        ||| `DeclRefExpr 0x23b28e0 <col:18> 'int' lvalue Var 0x23b2840 'i' 'int'
        ||| `IntegerLiteral 0x23b2900 <col:22> 'int' 10
      |-UnaryOperator 0x23b2978 <col:26, col:27> 'int' postfix '++'
        | `DeclRefExpr 0x23b2958 <col:26> 'int' lvalue Var 0x23b2840 'i' 'int'
      `CompoundStmt 0x23b2a18 <line:5:2, line:7:2>
        `CompoundAssignOperator 0x23b29e8 <line:6:3, col:11> 'int' lvalue '+=' ComputeLHSTy='int' ComputeResultTy='int'
          |-DeclRefExpr 0x23b2990 <col:3> 'int' lvalue Var 0x23b2788 'sum1' 'int'
          `ImplicitCastExpr 0x23b29d0 <col:11> 'int' <LValueToRValue>
            `DeclRefExpr 0x23b29b0 <col:11> 'int' lvalue Var 0x23b2840 'i' 'int'
      |-DeclStmt 0x23b2bc0 <line:9:2, col:21>
        | `VarDecl 0x23b2a80 <col:2, col:13> col:6 used sum2 'int' cinit
        |   `IntegerLiteral 0x23b2ae8 <col:13> 'int' 3
      `VarDecl 0x23b2b20 <col:2, col:20> col:16 used i 'int' cinit
        `IntegerLiteral 0x23b2b88 <col:20> 'int' 4
    |-WhileStmt 0x23b2d30 <line:10:2, line:14:2>
      |-BinaryOperator 0x23b2c30 <line:10:8, col:12> 'bool' '<'
        |||-ImplicitCastExpr 0x23b2c18 <col:8> 'int' <LValueToRValue>
        ||| `DeclRefExpr 0x23b2bd8 <col:8> 'int' lvalue Var 0x23b2b20 'i' 'int'
        ||| `IntegerLiteral 0x23b2bf8 <col:12> 'int' 10
      `CompoundStmt 0x23b2d10 <line:11:2, line:14:2>
        |-CompoundAssignOperator 0x23b2ca8 <line:12:3, col:11> 'int' lvalue '+=' ComputeLHSTy='int' ComputeResultTy='int'
          |||-DeclRefExpr 0x23b2c50 <col:3> 'int' lvalue Var 0x23b2a80 'sum2' 'int'
          ||| `ImplicitCastExpr 0x23b2c90 <col:11> 'int' <LValueToRValue>
          ||| `DeclRefExpr 0x23b2c70 <col:11> 'int' lvalue Var 0x23b2b20 'i' 'int'
          `UnaryOperator 0x23b2cf8 <line:13:3, col:4> 'int' postfix '++'
          `DeclRefExpr 0x23b2cd8 <col:3> 'int' lvalue Var 0x23b2b20 'i' 'int'
    `IfStmt 0x23b2eb0 <line:16:2, line:19:10> has_else
      |-BinaryOperator 0x23b2e00 <line:16:5, col:19> 'bool' '<'
        ||-BinaryOperator 0x23b2dc0 <col:5, col:12> 'int' '+'
        |||-ImplicitCastExpr 0x23b2d90 <col:5> 'int' <LValueToRValue>
        ||| `DeclRefExpr 0x23b2d50 <col:5> 'int' lvalue Var 0x23b2788 'sum1' 'int'
        ||| `ImplicitCastExpr 0x23b2da8 <col:12> 'int' <LValueToRValue>

```

```

||  `DeclRefExpr 0x23b2d70 <col:12> 'int' lvalue Var 0x23b2a80 'sum2' 'int'
|  `IntegerLiteral 0x23b2de0 <col:19> 'int' 100
|-ReturnStmt 0x23b2e58 <line:17:3, col:10>
|  `ImplicitCastExpr 0x23b2e40 <col:10> 'int' <LValueToRValue>
|  `DeclRefExpr 0x23b2e20 <col:10> 'int' lvalue Var 0x23b2788 'sum1' 'int'
|-ReturnStmt 0x23b2ea0 <line:19:3, col:10>
|  `ImplicitCastExpr 0x23b2e88 <col:10> 'int' <LValueToRValue>
|  `DeclRefExpr 0x23b2e68 <col:10> 'int' lvalue Var 0x23b2a80 'sum2' 'int'

```

Example 4: switch

```

int main()
{
    int a = 3;
    switch(a)
    {
        case 1: a = a + 1; break;
        case 3: a = a + 3;
        case 4: a = a + 4; break;
    }

    return a;
}

```

```

; ModuleID = './switch.cpp'
source_filename = "./switch.cpp"
target datalayout = "e-m:e-p270:32:32-p271:32:32-p272:64:64-i64:64-f80:128-
n8:16:32:64-S128"
target triple = "x86_64-pc-linux-gnu"

```

```

; Function Attrs: mustprogress noinline norecurse nounwind optnone uwtable
define dso_local noundef i32 @main() #0 {
    %1 = alloca i32, align 4
    %2 = alloca i32, align 4
    store i32 0, i32* %1, align 4
    store i32 3, i32* %2, align 4
    %3 = load i32, i32* %2, align 4
    switch i32 %3, label %13 [
        i32 1, label %4
        i32 3, label %7
        i32 4, label %10
    ]
}

```

```

4:                                     ; preds = %0
    %5 = load i32, i32* %2, align 4
    %6 = add nsw i32 %5, 1
    store i32 %6, i32* %2, align 4
    br label %13

7:                                     ; preds = %0
    %8 = load i32, i32* %2, align 4
    %9 = add nsw i32 %8, 3
    store i32 %9, i32* %2, align 4
    br label %10

10:                                    ; preds = %0, %7
    %11 = load i32, i32* %2, align 4
    %12 = add nsw i32 %11, 4
    store i32 %12, i32* %2, align 4
    br label %13

13:                                    ; preds = %0, %10, %4
    %14 = load i32, i32* %2, align 4
    ret i32 %14
}

```

```

attributes #0 = { mustprogress noinline norecurse nounwind optnone uwtable "frame-
pointer"="all" "min-legal-vector-width"="0" "no-trapping-math"="true" "stack-
protector-buffer-size"="8" "target-cpu"="x86-64" "target-
features"="+cx8,+fxsr,+mmx,+sse,+sse2,+x87" "tune-cpu"="generic" }

```

```

!llvm.module.flags = !{!0, !1, !2, !3, !4}
!llvm.ident = !{!5}

```

```

!0 = !{i32 1, !"wchar_size", i32 4}
!1 = !{i32 7, !"PIC Level", i32 2}
!2 = !{i32 7, !"PIE Level", i32 2}
!3 = !{i32 7, !"uwtable", i32 1}
!4 = !{i32 7, !"frame-pointer", i32 2}
!5 = !{"Ubuntu clang version 14.0.6"}

```



```

`-FunctionDecl 0x1f7f720 <switch.cpp:1:1, line:12:1> line:1:5 main 'int ()'
  |-CompoundStmt 0x1f7fd38 <line:2:1, line:12:1>
    |-DeclStmt 0x1f7f8d0 <line:3:2, col:11>
      |-VarDecl 0x1f7f848 <col:2, col:10> col:6 used a 'int' cinit
        |-IntegerLiteral 0x1f7f8b0 <col:10> 'int' 3
    |-SwitchStmt 0x1f7f920 <line:4:2, line:9:2>
      |-ImplicitCastExpr 0x1f7f908 <line:4:9> 'int' <LValueToRValue>
      |-DeclRefExpr 0x1f7f8e8 <col:9> 'int' lvalue Var 0x1f7f848 'a' 'int'
    |-CompoundStmt 0x1f7fcb8 <line:5:2, line:9:2>
      |-CaseStmt 0x1f7f988 <line:6:3, col:19>
        |-ConstantExpr 0x1f7f968 <col:8> 'int'
        |-value: Int 1
        |-IntegerLiteral 0x1f7f948 <col:8> 'int' 1
        |-BinaryOperator 0x1f7fa48 <col:11, col:19> 'int' lvalue '='
          |-DeclRefExpr 0x1f7f9b0 <col:11> 'int' lvalue Var 0x1f7f848 'a' 'int'
          |-BinaryOperator 0x1f7fa28 <col:15, col:19> 'int' '+'
            |-ImplicitCastExpr 0x1f7fa10 <col:15> 'int' <LValueToRValue>
            |-DeclRefExpr 0x1f7f9d0 <col:15> 'int' lvalue Var 0x1f7f848 'a' 'int'
            |-IntegerLiteral 0x1f7f9f0 <col:19> 'int' 1
        |-BreakStmt 0x1f7fa68 <col:22>
      |-CaseStmt 0x1f7fab0 <line:7:3, col:19>
        |-ConstantExpr 0x1f7fa90 <col:8> 'int'
        |-value: Int 3
        |-IntegerLiteral 0x1f7fa70 <col:8> 'int' 3
        |-BinaryOperator 0x1f7fb70 <col:11, col:19> 'int' lvalue '='
          |-DeclRefExpr 0x1f7fad8 <col:11> 'int' lvalue Var 0x1f7f848 'a' 'int'
          |-BinaryOperator 0x1f7fb50 <col:15, col:19> 'int' '+'
            |-ImplicitCastExpr 0x1f7fb38 <col:15> 'int' <LValueToRValue>
            |-DeclRefExpr 0x1f7faf8 <col:15> 'int' lvalue Var 0x1f7f848 'a' 'int'
            |-IntegerLiteral 0x1f7fb18 <col:19> 'int' 3
        |-CaseStmt 0x1f7fbd0 <line:8:3, col:19>
          |-ConstantExpr 0x1f7fbb0 <col:8> 'int'
          |-value: Int 4
          |-IntegerLiteral 0x1f7fb90 <col:8> 'int' 4
          |-BinaryOperator 0x1f7fc90 <col:11, col:19> 'int' lvalue '='
            |-DeclRefExpr 0x1f7fbf8 <col:11> 'int' lvalue Var 0x1f7f848 'a' 'int'
            |-BinaryOperator 0x1f7fc70 <col:15, col:19> 'int' '+'
              |-ImplicitCastExpr 0x1f7fc58 <col:15> 'int' <LValueToRValue>
              |-DeclRefExpr 0x1f7fc18 <col:15> 'int' lvalue Var 0x1f7f848 'a' 'int'
              |-IntegerLiteral 0x1f7fc38 <col:19> 'int' 4
          |-BreakStmt 0x1f7fcb0 <col:22>
    |-ReturnStmt 0x1f7fd28 <line:11:2, col:9>
      |-ImplicitCastExpr 0x1f7fd10 <col:9> 'int' <LValueToRValue>
      |-DeclRefExpr 0x1f7fcf0 <col:9> 'int' lvalue Var 0x1f7f848 'a' 'int'

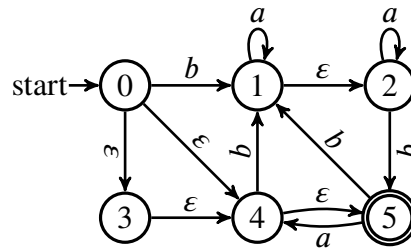
```

武汉大学计算机学院
2022 - 2023 学年第 1 学期
《编译原理》期末考试试卷 (A)
[闭卷, 试卷共 4 页, 答题时间 120 分钟]

学号: _____ 姓名: _____ 专业: _____ 成绩: _____

注意: (1) 请将答案一律写在答卷纸上, 注意标明题号并按顺序解答。答在试卷上不得分。
(2) 离开考场时将试卷及答卷纸一起上交。

一、 设 NFA N 的状态转换图如下所示: (25 分, 每小题 5 分)



- (1) 试写出 NFA N 接受字符串 “baabaabba” 的过程;
- (2) 设用子集构造法求出的与 NFA N 等价的 DFA M 有 4 个状态 A, B, C 和 D , 其中 $A = \varepsilon\text{-closure}(\{0\})$, 状态转换函数 $\text{Dtrans}(A, a) = B$, $\text{Dtrans}(A, b) = C$ 试求与状态 A, B, C 和 D 所对应的 NFA N 的状态集, 并画出 DFA M 的状态转换图;
- (3) 求 DFA M 的最小状态自动机;
- (4) 试用自然语言描述 NFA N 所接受的语言;
- (5) 求正则表达式 r , 使得 $L(r) = L(N)$.

二、 设文法 $G(D)$ 定义如下:

$$D \rightarrow DD \mid n[D] \mid \text{id}$$

其中: ‘id’, ‘[’, ‘]’, 和 ‘n’ 为终结符, D 是文法开始符号. (25 分, 每小题 5 分)

- (1) 试写出语句 “n[n[id]]n[id]” 的一个最左推导;
- (2) 试消除文法 $G(D)$ 中的左递归;
- (3) 试对消除左递归后的文法所有非终结符求 First 集和 Follow 集;
- (4) 试对消除左递归后的文法构造 LL(1) 分析表, 从而说明消除左递归后的文法不是 LL(1) 文法;
- (5) 试利用你的分析表写出语句 “n[n[id]]” 的一个正确的分析过程.

三、 设文法 $G(D)$ 如题二所示：

(10 分, 每小题 5 分)

- (1) 试对语句 “id n[id]id” 画出两棵不同的语法树，从而说明该文法为二义文法；
- (2) 试设计一个与文法 $G(D)$ 等价的无二义的文法，使得运算 ‘ DD ’ 为右结合.

四、 设题二文法 $G(D)$ 的拓广文法 $G(D')$ 如下所示：

(20 分, 每小题 5 分)

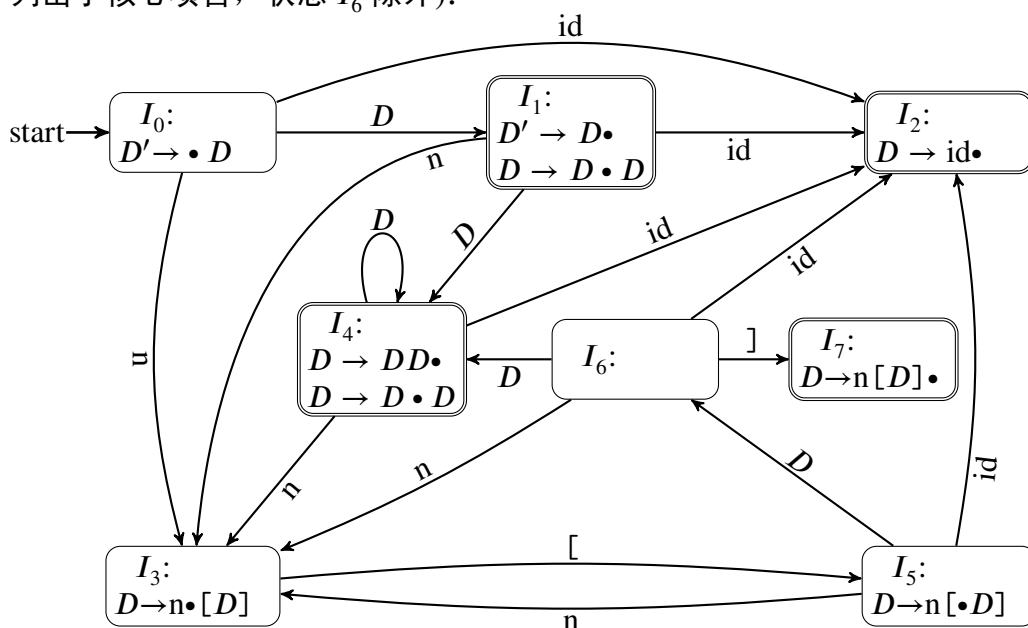
$$D' \rightarrow D \quad (0)$$

$$D \rightarrow DD \quad (1)$$

$$D \rightarrow n[D] \quad (2)$$

$$D \rightarrow id \quad (3)$$

文法 $G(D')$ 的识别活前缀 LR(0) 项目自动机 M 如下图所示 (注意每个状态仅列出了核心项目，状态 I_6 除外)：



- (1) 试求状态 I_6 所对应的 LR(0) 项目集；
- (2) 用该识别活前缀的自动机分析输入串 “ $(n[])^m(id)^m n id$ ” ($m \geq 1$). 试问在输入串移进到何处报错, 在报错前最多进行了多少次归约；
- (3) 试构造该文法的 SLR 分析表，并对分析表中的移进/归约和归约/归约冲突选择正确的移进或归约动作，使得文法 $G(D)$ 的所有语句能被正确地分析且运算的结合次序与题三所规定的一致；
- (4) 试利用你的分析表写出 “n[id]id” 的分析过程.

五、设题二文法 $G(D)$ 所生成的语句编码字符串. 设终结符 n 的属性 $n.int$ 为 n 所对应的非零正整数, 属性 $id.string$ 为 id 对应的词形, 属性 $D.decoded$ 为解码后的字符串. 其解码规则为: $n[D]$ 的解码后的字符串为 $(D.decoded)^{n.int}$.
 如: (10 分, 每小题 5 分)

序号	编码串	解码串
1	abc	abc
2	2[abc]	abcabc
3	3[2[abc]d]	abcabcdnabcdnabcdnabcdn

- (1) 试为文法 $G(D)$ 设计解码的语法制导定义;
- (2) 试为语句 “2[a3[b4[c]]d]” 解码.

六、设有如下 Pascal 程序片段: (5 分)

```
while ((a > b) or not (c > d)) do
begin
    if not (e > f) and (i > j) then
        x := x + 1;
    else
        break;
    if not (m > n) then continue;
    y := y + 2;
end.
```

其对应的三地址码如下所示

```
L0: [ ] (a > b) goto L__ | [ ] (m > n) goto L__
    [ ] (c > d) goto L__ |      t1 := y + 2
L1: [ ] (e > f) goto L__ |      y := t1
    [ ] (i > j) goto L__ |      goto L0
    t0 := x + 1           | L2:
    x := t0               |
```

试为其中空白 “__” 填上正确的标号编号, 并为空白 “[]” 填上 if 或 ifnot.

第七题见下页!

七、 设有如下 C 语言程序:

(5 分)

```
1  #include <stdio.h>
2  int main(void)
3  {
4      char *words[] = {"word1", "word2", "word3"};
5      char **ptr;
6      int i = 0;
7
8      *ptr = words[0];
9
10     while( i != 3) {
11         printf("%s", *ptr);
12         ptr++;
13         i++;
14     }
15
16     return 0;
17 }
```

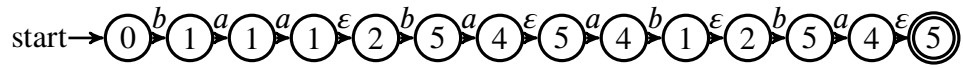
该程序在 Intel x86/Linux 下用 gcc 编译虽然有警告, 但能生成执行程序. 运行编译后的程序报错如下:

Segmentation fault (core dumped)

并未输出 “word1word2word3”. 试分析产生运行错误的原因.

武汉大学计算机学院 2022 - 2023 学年第一学期
《Compiler》(期末考试参考答案 A)

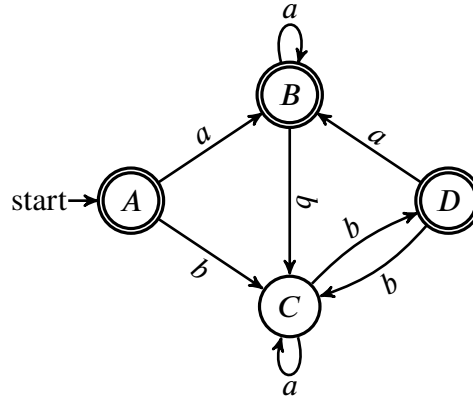
一、 (1)



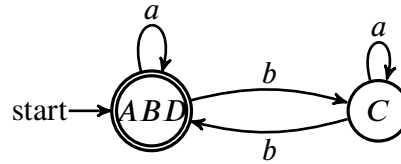
(2)

$$A = \{0, 3, 4, 5\}, B = \{4, 5\}, C = \{1, 2\}, D = \{5\}.$$

状态转换图为:



(3) 最小 DFA 如下所示:



(4) b 出现偶数次.

(5) $r = (a^*ba^*b)^*a^*$ 或 $a^*(ba^*ba^*)^*$.

二、 (1) 语句 “ $n[n[id]]n[id]$ ” 的最左推导如下:

$$\begin{array}{lll} D & \xRightarrow{lm} DD & \xRightarrow{lm} n[n[id]]D \\ & \xRightarrow{lm} n[D]D & \xRightarrow{lm} n[n[id]]n[D] \\ & \xRightarrow{lm} n[n[D]]D & \xRightarrow{lm} n[n[id]]n[id] \end{array}$$

(2) 消除左递归后的文法如下:

$$\begin{array}{l} D \rightarrow n[D]D' \mid id D' \\ D' \rightarrow D D' \mid \epsilon \end{array}$$

(3)

非终结符	First	Follow
D	id, n	id, n,], \$
D'	ϵ , id, n	id, n,], \$

(4) LL(1) 分析表如下所示

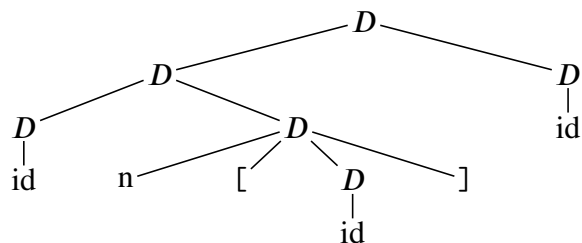
	id	n	[	]	\$
D	$D \rightarrow \text{id}D'$	$D \rightarrow n[D]D'$			
D'	$D' \rightarrow DD' \mid \epsilon$	$D' \rightarrow DD' \mid \epsilon$		$D' \rightarrow \epsilon$	$D' \rightarrow \epsilon$

(5) 语句 “n[n[id]]” 的分析过程如下所示：

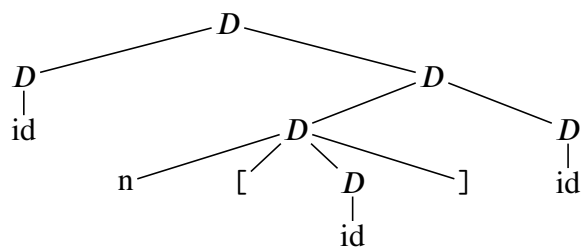
剩余串	分析栈	分析动作
n[n[id]]\$	$D\$$	$D \rightarrow n[D]D'$
n[n[id]]\$	$n[D]D'\$$	match-advance
[n[id]]\$	$[D]D'\$$	match-advance
n[id]]\$	$D]D'\$$	$D \rightarrow n[D]D$
n[id]]\$	$n[D]D']D'\$$	match-advance
[id]]\$	$[D]D']D'\$$	match-advance
id]]\$	$D]D']D'\$$	$D \rightarrow \text{id}D'$
id]]\$	$\text{id}D']D']D'\$$	match-advance
]]\$	$D']D']D'\$$	$D' \rightarrow \epsilon$
]]\$	$]D']D'\$$	match-advance
]\$	$D']D'\$$	$D' \rightarrow \epsilon$
]\$	$]D'\$$	match-advance
\$	$D'\$$	$D' \rightarrow \epsilon$
\$	\$	分析成功

三、 (1) 语句 “id n[id]id” 的两棵不同的语法树为：

语法树 1:



语法树 2:



(2) 无二义文法:

$$D \rightarrow n[D]D \mid \text{id}D \mid n[D] \mid \text{id}$$

四、 (1) 状态 I_6 的 LR(0) 项目集为

$$\overline{\{ D \rightarrow D \bullet D, D \rightarrow n[D \bullet] \}} \\ = \{ D \rightarrow D \bullet D, D \rightarrow n[D \bullet], D \rightarrow \bullet DD, D \rightarrow \bullet n[D], D \rightarrow \bullet id \}$$

(2) 移进 $(n[])^m$ 到状态 I_5 , 移进第一个 id 到状态 I_2 , 第一次归约后 goto 到状态 I_6 . 面对第二个 id 移进到状态 I_2 再归约并 goto 到状态 I_4 . 状态 I_4 有移进/归约冲突. 为了得到最大归约, 采用归约优先, 故归约并 goto 到状态 I_6 . 第三个 id 到第 m 个 id 都是同样经历两次归约后回到状态 I_6 . 到此已移进 $(n[])^m id^m$ 到状态 I_6 , 并归约了 $2(m-1)+1$ 次. I_6 移进 ‘n’ 之后到状态 I_3 , 而状态 I_3 没有 id 出边, 故报错.

(3) $\text{Follow}(D) = \{ id, n,], \$ \}$ 状态 I_4 面对 ‘id’ 和 ‘n’ 有移进/归约冲突. 分析表如下所示:

状态	action					goto
	id	n	[	]	\$	D
0	s2	s3				1
1	s2	s3			acc	4
2	r3	r3		r3	r3	
3			s5			
4	s2	s3		r1	r1	4
5	s2	s3				6
6	s2	s3		s7		4
7	r2	r2		r2	r2	

(4) “ $n[id]id$ ” 的分析过程如下所示:

剩余串	分析栈	分析动作
$n[id]id\$$	0	shift
$[id]id\$$	0n3	shift
$id]id\$$	0n3[5	shift
$]id\$$	0n3[5id2	reduce $D \rightarrow id$
$]id\$$	0n3[5D6	shift
$id\$$	0n3[5D6]7	reduce $D \rightarrow n[D]$
$id\$$	0D1	shift
$\$$	0D1id2	reduce $D \rightarrow id$
$\$$	0D1D4	reduce $D \rightarrow DD$
$\$$	0D1	accept

五、

(1) 语义规则如下:

产生式	语义规则
$D \rightarrow D_1 D_2$	$D.\text{decoded} = D_1.\text{decoded} + D_2.\text{decoded}$
$D \rightarrow n[D_1]$	$D.\text{decoded} = (D_1.\text{decoded})^{n.\text{int}}$
$D \rightarrow \text{id}$	$D.\text{decoded} = \text{id.string}$

(2) 语句“2[a3[b4[c]]d]”解码为: abccccbccccbccccdabccccbccccbccccd.

六、

```
L0:  if (a > b) goto L1      |      ifnot (m > n) goto L0
      if (c > d) goto L2      |      t1 := y + 2
L1:  if (e > f) goto L2      |      y := t1
      ifnot (i > j) goto L2   |      goto L0
      t0 := x + 1             |  L2:
      x := t0
```

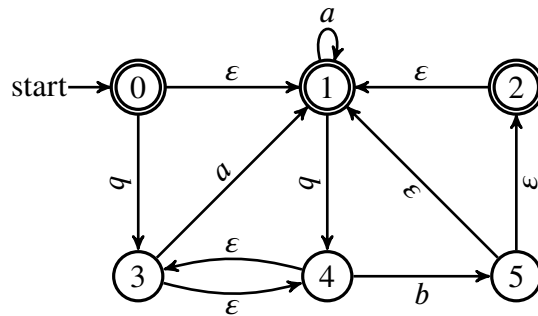
七、 指针 ptr 未初始化.

武汉大学计算机学院
2021 - 2022 学年第 2 学期
《编译原理》期末考试试卷 (A)
[闭卷, 试卷共 4 页, 答题时间 120 分钟]

学号: _____ 姓名: _____ 专业: _____ 成绩: _____

注意: (1) 请将答案一律写在答卷纸上, 注意标明题号并按顺序解答。答在试卷上不得分。
(2) 离开考场时将试卷及答卷纸一起上交。

一、 设 NFA N 的状态转换图如下所示: (25 分, 每小题 5 分)



- (1) 试写出 NFA N 接受字符串 “ababa” 的一个过程;
- (2) 设用子集构造法求出的与 NFA N 等价的 DFA M 有 4 个状态 A, B, C 和 D , 其中 $A = \epsilon\text{-closure}(\{0\})$, $D\text{trans}(A, a) = B$, $D\text{trans}(A, b) = C$ 试求与状态 A, B, C 和 D 所对应的 NFA N 的状态集, 并画出 DFA M 的状态转换图;
- (3) 求 DFA M 的最小状态自动机;
- (4) 试用自然语言描述 NFA N 所接受的语言;
- (5) 求正规表达式 r , 使得 $L(r) = L(N)$.

二、 设含有解引用和函数调用的 C 语言表达式文法 $G(E)$ 定义如下:

$$E \rightarrow *E \mid E(E) \mid (E) \mid \text{id}$$

其中: ‘*’, ‘(’, ‘)’, 和 ‘id’ 为终结符, E 是文法开始符号. (25 分, 每小题 5 分)

- (1) 试写出语句 “(*id)(id)(id)” 的一个最左推导;
- (2) 试消除文法 $G(E)$ 中的左递归;
- (3) 试对消除左递归后的文法所有非终结符求 First 集和 Follow 集;
- (4) 试对消除左递归后的文法构造 LL(1) 分析表, 从而说明消除左递归后的文法不是 LL(1) 文法;
- (5) 试利用你的分析表写出语句 “id(*id)” 的一个正确的分析过程.

三、 设文法 $G(E)$ 如题二所示：

(10 分, 5+5)

- (1) 试对语句 “*id(id)” 画出两棵不同的语法树，从而说明该文法为二义文法；
- (2) 试设计一个与文法 $G(E)$ 等价的无二义的文法，使得函数调用运算优先于解引用运算。

四、 设题二文法 $G(E)$ 的拓广文法 $G(E')$ 如下所示：

(20 分, 5+5+5+5)

$$E' \rightarrow E \quad (0)$$

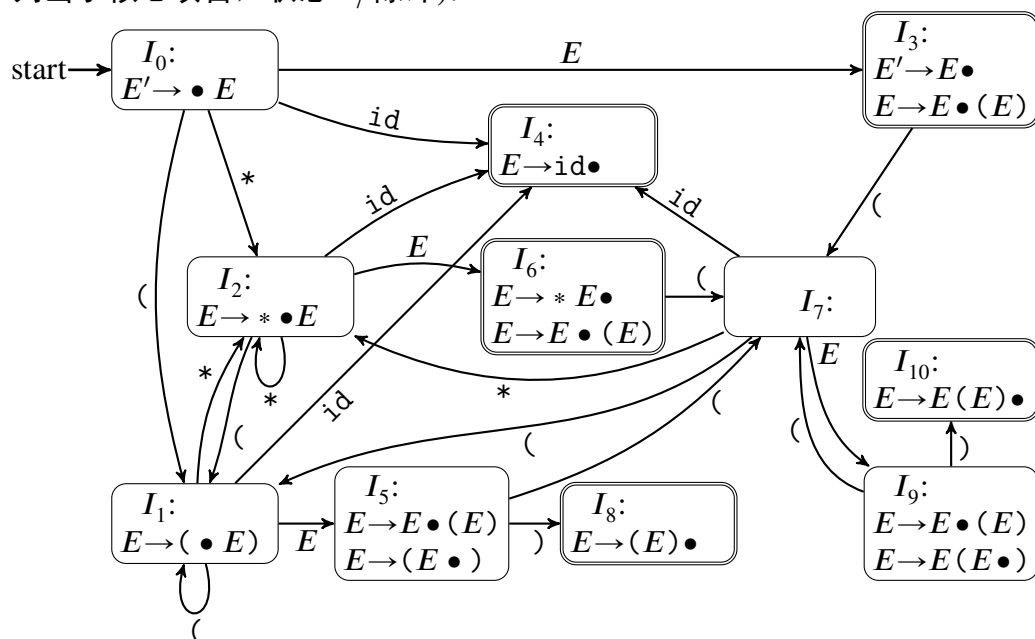
$$E \rightarrow *E \quad (1)$$

$$| E(E) \quad (2)$$

$$| (E) \quad (3)$$

$$| id \quad (4)$$

文法 $G(E')$ 的识别活前缀 LR(0) 项目自动机 M 如下图所示 (注意每个状态仅列出了核心项目，状态 I_7 除外)：



- (1) 试求状态 I_7 所对应的 LR(0) 项目集；
- (2) 试求活前缀 “ $((**)^n E$ ” ($n \geq 1$) 至少要经过多少次归约才能把整个前缀归约到 E ，为什么？
- (3) 试构造该文法的 SLR 分析表，并对分析表中的移进/归约和归约/归约冲突选择正确的移进或归约动作，使得文法 $G(E)$ 的所有语句能被正确地分析且运算的优先级与结合次序与题三所规定的一致；
- (4) 试利用你的分析表写出 “*id(id)” 的分析过程。

五、现需化简题二文法 $G(E)$ 所生成的 C 语言表达式，即尽可能地删除括号对，并使得化简后的表达式按题三规定的运算优先级结合后与原表达式运算次序一致。如： (10 分, 5+5)

序号	C 语言表达式	最简表达式
1	$(*(id(id)))$	$*id(id)$
2	$((**id)((id)))$	$(**id)(id)$

为此设计为非终结符 E 设计综合属性 exp ，其值为该非终结符所表示的语法成分对应的最简表达式；综合属性 $E.is_star$ 取值为布尔量 $\{true, false\}$ 。当 E 表示的表达式最后一次运算为解引用时，其取值为 $true$ ，否则为 $false$ 。 $id.lexeme$ 取值为 id 对应的字符串。

- (1) 试设计将文法 $G(E)$ 生成的 C 语言表达式转换成最简表达式的 SDD；
- (2) 试写出表达式 “ $((**((*(a)(b))(c)(d)(e)))(h))$ ” 的最简表达式。

六、设有如下 Pascal 程序片段： (5 分)

```
while a > b and not (c > d) do
begin
  x := x + 1;
  if e > f and g > h or not (i > k) then continue;
  else x := y + 2;
end;
```

其对应的三地址码如下所示

```
L1: [ ] (a > b) goto L__ | L0: [ ] (i > k) goto L__
    [ ] (c > d) goto L__ |      t0 := y + 2
    x := x + 1           |      x := t0
    [ ] (e > f) goto L__ |      goto L__
    [ ] (g > h) goto L__ | L2:
```

试为其中空白 “__” 填上正确的标号编号，并为空白 “[]” 填上 `if` 或 `ifnot`。

第七题见下页！

七、 设有如下 C 语言程序:

(5 分)

```
1  #include <stdio.h>
2
3  void init(char *(*p)[3]) {
4      static char *s[] = {"foo", "bar", "baz"};
5      p = &s;
6  }
7
8  int main()
9  {
10     char *(*p)[3];
11     init(p);
12     for (int i=0; i<3; i++) printf("%s", (*p)[i]);
13     return 0;
14 }
```

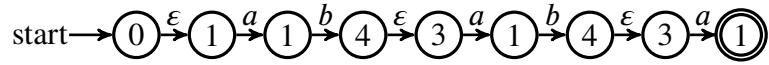
该程序在 Intel x86/Linux 下用 gcc 编译没有任何警告, 但运行编译后的程序报错如下:

Segmentation fault (core dumped)

试分析产生运行错误的原因. 若将代码第 11 行修改为 `init(&p)`, 试问如何修改函数 `init()`, 使得程序能输出 `foobarbaz` 并正常退出.

武汉大学计算机学院 2021 - 2022 学年第二学期
《编译原理》(期末考试参考答案 A)

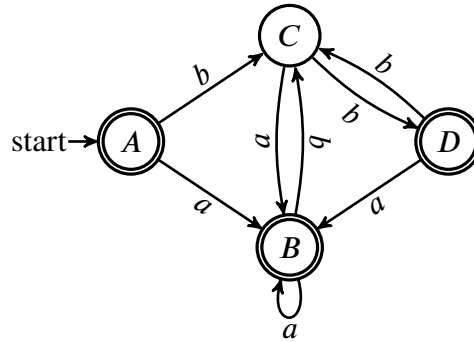
一、 (1)



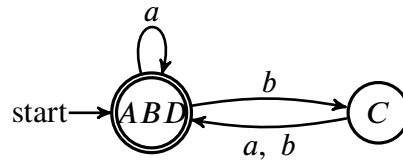
(2)

$$A = \{0, 1\}, B = \{1\}, C = \{3, 4\}, D = \{1, 2, 5\}.$$

状态转换图为:



(3) 最小 DFA 如下所示:



(4) 不以奇数 b 结尾.

(5) $r = (bb \mid ba \mid a)^*$.

二、 (1) 语句 “ $(\ast id)(id)(id)$ ” 的一个最左推导如下:

$$\begin{array}{lll} E & \xRightarrow{lm} & E(E) \\ & \xRightarrow{lm} & E(E)(E) \\ & \xRightarrow{lm} & (E)(E)(E) \\ & \xRightarrow{lm} & (\ast E)(E)(E) \end{array} \quad \begin{array}{l} \xRightarrow{lm} (\ast id)(E)(E) \\ \xRightarrow{lm} (\ast id)(id)(E) \\ \xRightarrow{lm} (\ast id)(id)(id) \end{array}$$

(2) 消除左递归后的文法如下:

$$\begin{array}{l} E \rightarrow \ast EE' \mid (E)E' \mid idE' \\ E' \rightarrow (E)E' \mid \varepsilon \end{array}$$

(3)

非终结符	First	Follow
E	id, (, *	(,), \$
E'	ε , (	(,), \$

(4) LL(1) 分析表如下所示

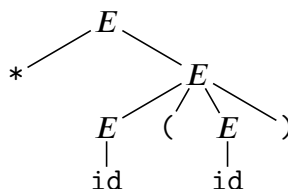
	id	(	)	*	\$
E	$E \rightarrow idE'$	$E \rightarrow (E)E'$		$E \rightarrow *EE'$	
E'		$E' \rightarrow (E)E' \mid \epsilon$	$E' \rightarrow \epsilon$		$E' \rightarrow \epsilon$

(5) 语句 “id(*id)” 的分析过程如下所示:

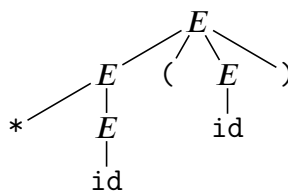
剩余串	分析栈	分析动作
id(*id)\$	$E\$$	$E \rightarrow idE'$
id(*id)\$	$idE'\$$	match-advance
(*id)\$	$E'\$$	$E' \rightarrow (E)E'$
(*id)\$	$(E)E'\$$	match-advance
*id)\$	$E)E'\$$	$E \rightarrow *EE'$
*id)\$	$*EE')E'\$$	match-advance
id)\$	$EE')E'\$$	$E \rightarrow idE'$
id)\$	$idE'E')E'\$$	match-advance
)\$	$E'E')E'\$$	$E' \rightarrow \epsilon$
)\$	$E')E'\$$	$E' \rightarrow \epsilon$
)\$	$)E'\$$	match-advance
\$	$E'\$$	$E' \rightarrow \epsilon$
\$	\$	分析成功

三、 (1) 语句 “*id(id)” 的两棵不同的语法树为:

语法树 1:



语法树 2:



(2) 无二义文法:

$$\begin{aligned}
 E &\rightarrow *E \mid T \\
 T &\rightarrow T(E) \mid (E) \mid id
 \end{aligned}$$

四、 (1) 状态 I_7 的 LR(0) 项目集为

$$\begin{aligned}
 &\overline{\{E \rightarrow E(\bullet E)\}} \\
 &= \{E \rightarrow E(\bullet E), E \rightarrow \bullet *E, E \rightarrow \bullet E(E), \\
 &\quad E \rightarrow \bullet (E), E \rightarrow \bullet id\}
 \end{aligned}$$

(2)

$$\begin{array}{lcl}
 E & \xRightarrow{rm} & (E) \\
 & \xRightarrow{rm} & (*E) \\
 & \xRightarrow{rm} & (**E) \\
 & \xRightarrow{rm} & (**(E)) \\
 & \xRightarrow{rm} & (**(*E)) \\
 & \xRightarrow{rm} & (**(**E)) \\
 & \dots & \\
 & \xRightarrow{rm} & ((**)^{n-1}(E))^{n-1} \\
 & \xRightarrow{rm} & ((**)^{n-1}(*E))^{n-1} \\
 & \xRightarrow{rm} & ((**)^{n-1}(**E))^{n-1}
 \end{array}
 \left. \begin{array}{l} \\ \\ \\ \\ \\ \end{array} \right\} \begin{array}{l} 3\text{次} \\ 3\text{次} \\ 3\text{次} \end{array}$$

故需至少归约 $3n$ 次.

(3) $\text{Follow}(E) = \{ (,), \$ \}$. 状态 I_6 面对 ‘(’ 有移进/归约冲突. 分析表如下所示:

状态	action					goto
	id	(	)	*	\$	E
0	s4	s1		s2		3
1	s4	s1		s2		5
2	s4	s1		s2		6
3		s7			acc	
4		r4	r4		r4	
5		s7	s8			
6		s7	r1		r1	
7	s4	s1		s2		9
8		r3	r3		r3	
9		s7	s10			
10		r2	r2		r2	

(4) “*id(id)” 的分析过程如下所示:

剩余串	分析栈	分析动作
*id(id)\$	0	shift
id(id)\$	0*2	shift
(id)\$	0*2id4	reduce $E \rightarrow id$
(id)\$	0*2E6	shift
id)\$	0*2E6(7	shift
)\$	0*2E6(7id4	reduce $E \rightarrow id$
)\$	0*2E6(7E9	shift
\$	0*2E6(7E9)10	reduce $E \rightarrow E(E)$
\$	0*2E6	reduce $E \rightarrow *E$
\$	0E3	accept

五、 (1) 解

产生式	语义规则
$E \rightarrow *E_1$	$E.exp = "*" + E_1.exp$ $E.is_star = True$
$E \rightarrow E_1(E_2)$	if $E_1.is_star$ $E.exp = "(" + E_1.exp + ")" (" + E_2.exp + ")"$ else $E.exp = E_1.exp + "(" + E_2.exp + ")"$ $E.is_star = False$
$E \rightarrow (E_1)$	$E.exp = E_1.exp$ $E.is_star = E_1.is_star$
$E \rightarrow id$	$E.exp = id.lexeme$ $E.is_star = False$

(2) $((**(*a(b)))(c)(d)(e))(h).$

六、

```

L1: ifnot (a > b) goto L2 | L0: ifnot (i > k) goto L1
    if (c > d) goto L2    |    t0 := y + 2
    x := x + 1            |    x := t0
    ifnot (e > f) goto L0 |    goto L1
    if (g > h) goto L1    | L2:

```

七、 由于 C 语言采用传值方式调用, `init(p)` 并未对 `main()` 中的 `p` 赋值. 程序 11 行修改为 `init(&p)`, `init()` 应修改为

```

3 void init(char (**p)[3]) {
5     *p = &s;

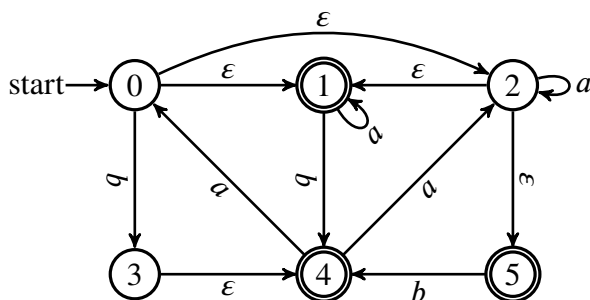
```

武汉大学计算机学院
2019 - 2020 学年第一学期 2018 级
《编译原理》期末考试试卷 (A)

学号: _____ 姓名: _____ 专业: _____ 成绩: _____

(注: ①考试时间为 120 分钟; ②所有的解答必须写在答题纸上, 并注明题号。)

一、 设 NFA N 的状态转换图如下所示: (25 分, 每小题 5 分)



- (1) 试写出 NFA N 接受字符串 “baaba” 的一个过程;
- (2) 设用子集构造法求出的与 NFA N 等价的 DFA M 有 4 个状态 A, B, C 和 D , 其中 $A = \epsilon\text{-closure}(\{0\})$, $Dtrans(A, a) = B$, $Dtrans(A, b) = C$ 试求与状态 A, B, C 和 D 所对应的 NFA N 的状态集, 并画出 DFA M 的状态转换图;
- (3) 求 DFA M 的最小状态自动机;
- (4) 试用自然语言描述 NFA N 所接受的语言;
- (5) 求正规表达式 r , 使得 $L(r) = L(N)$.

二、 设含有否定 $\neg$ 和蕴含 $\rightarrow$ 运算的命题公式文法 $G(F)$ 定义如下:

$$F \rightarrow F \rightarrow F \mid \neg F \mid (F) \mid a$$

其中: ‘a’, ‘ $\rightarrow$ ’, ‘ $\neg$ ’, ‘(’ 和 ‘)’ 为终结符, F 是文法开始符号. (25 分, 每小题 5 分)

- (1) 试写出语句 “ $\neg(a \rightarrow a)$ ” 的一个最左推导;
- (2) 试消除文法 $G(F)$ 中的左递归和左公因子;
- (3) 试对消除左递归后的文法所有非终结符求 First 集和 Follow 集;
- (4) 试对消除左递归后的文法构造 LL(1) 分析表, 从而说明消除左递归后的文法不是 LL(1) 文法;
- (5) 试利用你的分析表写出语句 “ $\neg(a \rightarrow a)$ ” 的一个正确的分析过程.

三、 设文法 $G(F)$ 如题二所示：

(10 分, 5+5)

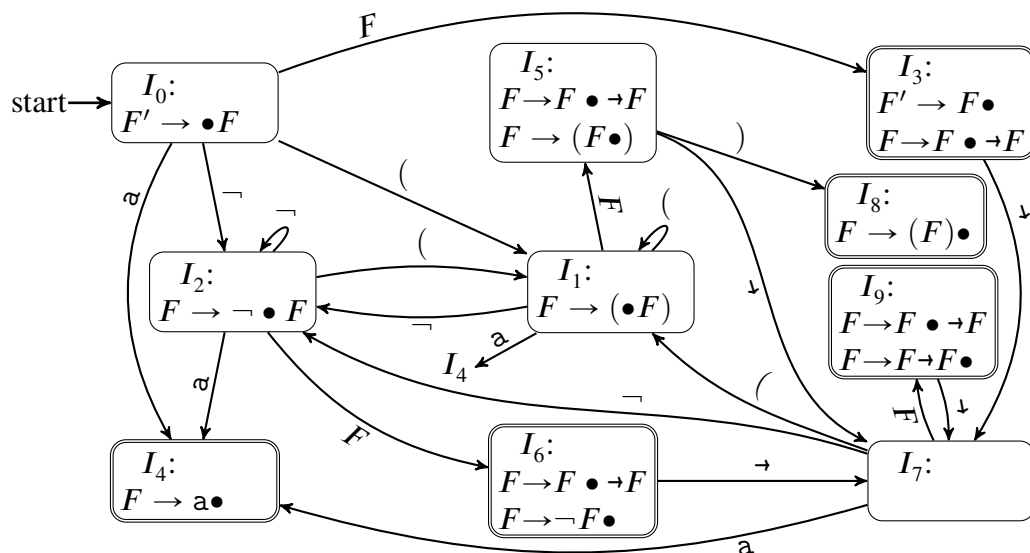
- (1) 试对语句 “ $\neg a \rightarrow a$ ” 画出两棵不同的语法树，从而说明该文法为二义文法；
- (2) 试设计一个与文法 $G(F)$ 等价的无二义的文法，使得蕴含 $(F \rightarrow F)$ 为右结合，且其的优先级低于否定。

四、 设题二文法 $G(F)$ 的拓广文法 $G(F')$ 如下所示：

(20 分, 5+5+5+5)

$$\begin{aligned}
 F' &\rightarrow F & (0) \\
 F &\rightarrow F \rightarrow F & (1) \\
 &| \neg F & (2) \\
 &| (F) & (3) \\
 &| a & (4)
 \end{aligned}$$

文法 $G(F')$ 的识别活前缀 LR(0) 项目自动机 M 如下图所示 (注意每个状态仅列出了核心项目，状态 I_7 除外)：



- (1) 试求状态 I_7 所对应的 LR(0) 项目集；
- (2) 试求仅由终结符号组成的活前缀对应的正则表达式；
- (3) 试构造该文法的 SLR 分析表，并对分析表中的移进/归约和归约/归约冲突选择正确的移进或归约动作，使得文法 $G(F)$ 的所有语句能被正确地分析且运算的优先级与结合次序与题三所规定的一致；
- (4) 试利用你的分析表写出语句 “ $\neg a \rightarrow a$ ” 的分析过程。

五、现需对题四文法 $G(F')$ 所生成的命题公式转换为析取 ($\vee$)、合取 ($\wedge$) 和仅有对原子取否的逻辑等价公式，如：

序号	原命题公式	转换后的命题公式
1	$A \rightarrow B$	$\neg A \vee B$
2	$\neg \neg A$	A
3	$\neg (A \rightarrow B)$	$(A) \wedge (\neg B)$
4	$A \rightarrow (B \rightarrow C)$	$\neg A \vee \neg B \vee C$
5	$(A \rightarrow B) \rightarrow C$	$(A) \wedge (B) \vee C$

为此设计继承属性 $F.is_neg$, 其取值为布尔量 `True` 和 `False`; 综合属性 $F.nnf$, 其取值为 F 所表示的语法成分对应的转换后的命题公式 (字符串); $a.lexeme$ 取值为 a 所对应的字符串. $F.is_neg$ 的语义规则如下所示: (10 分, 5+5)

产生式	语义规则
$F' \rightarrow F$	$F.is_neg = \text{False}$
$F \rightarrow F_1 \rightarrow F_2$	$F_1.is_neg = \neg F.is_neg; F_2.is_neg = F.is_neg$
$F \rightarrow \neg F_1$	$F_1.is_neg = \neg F.is_neg$
$F \rightarrow (F_1)$	$F_1.is_neg = F.is_neg$

- (1) 试写出属性 $F.nnf$ 的语法制导定义;
- (2) 试求 “ $\neg(((A \rightarrow \neg B) \rightarrow C) \rightarrow \neg(D \rightarrow E))$ ” 转换后的命题公式.

六、设有如下 Pascal 程序片段: (5 分)

```
repeat
  if g < h or not i < j then continue;
  else x := y + 1;
  z := x + y;
until not (not (a < b) or c < d and e < f);
```

其对应的三地址码如下所示

```
L0: [   ] (g<h) goto L__ |      z := t1
    [   ] (i<j) goto L__ | L1: [   ] (a<b) goto L__
    t0 := y+1             |      [   ] (c<d) goto L__
    x := t0               |      [   ] (e<f) goto L__
    t1 := x+y             | L2:
```

试为其中空白 “`__`” 填上正确的标号编号, 并为空白 “[]” 填上 `if` 或 `ifnot`.

第七题见下页!

七、 设有如下 C 语言程序：

(5 分)

```
1  #include <stdio.h>
2
3  void init(char **s)
4  {
5      char a[][8] = {"Stay", "Safe", "From", "Covid"};
6      for (int i = 0; i < 4; i++)
7          s[i] = a[i];
8  }
9
10 int main()
11 {
12     char *s[4];
13     init(s);
14     for (int i = 0; i < 4; i++)
15         printf("%s\n", s[i]);
16     return 0;
17 }
```

该程序在 Intel x86/Linux 下用 gcc 编译没有任何警告，但运行编译后的程序输出乱码：

Hã}HÉJ IH ...

但若把第 5 行修改为：

```
char *a[] = {"Stay", "Safe", "From", "Covid"};
```

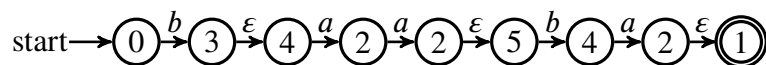
则程序可正确地输出：

StaySafeFromCovid

试分析产生运行错误的原因。

武汉大学计算机学院 2020 - 2021 学年第一学期
2018 级《编译原理》(期末考试参考答案 A)

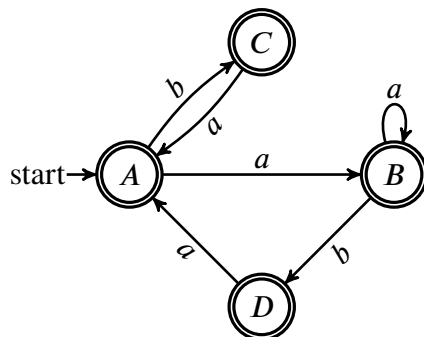
一、 (1)



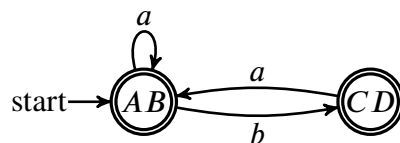
(2)

$$A = \{0, 1, 2, 5\}, B = \{1, 2, 5\}, C = \{3, 4\}, D = \{4\}.$$

状态转换图为:



(3) 最小 DFA 如下所示:



(4) 无连续的 b .

(5) $(b \mid \epsilon)(a \mid ab)^*$.

二、 (1) 语句 “ $\neg(a \rightarrow a)$ ” 的最左推导如下:

$$\begin{array}{lll} F & \xRightarrow{lm} & \neg F \\ & \xRightarrow{lm} & \neg(F) \\ & \xRightarrow{lm} & \neg(F \rightarrow F) \end{array} \quad \begin{array}{lll} & \xRightarrow{lm} & \neg(a \rightarrow F) \\ & \xRightarrow{lm} & \neg(a \rightarrow a) \end{array}$$

(2) 消除左递归和左公因子后的文法如下:

$$\begin{array}{l} F \rightarrow \neg F F' \mid a F' \mid (F) F' \\ F' \rightarrow \rightarrow F F' \mid \epsilon \end{array}$$

(3)

非终结符	First	Follow
F	$a, (, \neg$	$\neg,), \$$
F'	$\neg, \epsilon$	$\neg,), \$$

(4) LL(1) 分析表如下所示

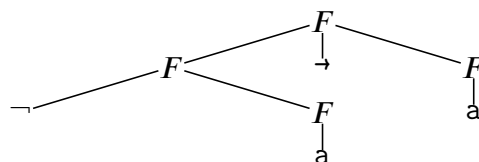
	a	$\neg$	$\rightarrow$	(	)	\$
F	$F \rightarrow aF'$	$F \rightarrow \neg FF'$		$F \rightarrow (F)F'$		
F'			$F' \rightarrow \rightarrow FF' \mid \epsilon$		$F' \rightarrow \epsilon$	$F' \rightarrow \epsilon$

(5) 语句 “ $\neg(a \rightarrow a)$ ” 的分析过程如下所示：

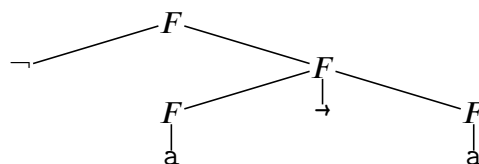
剩余串	分析栈	分析动作
$\neg(a \rightarrow a)\$$	$F\$$	$F \rightarrow \neg FF'$
$\neg(a \rightarrow a)\$$	$\neg FF'\$$	match-advance
$(a \rightarrow a)\$$	$FF'\$$	$F \rightarrow (F)F'$
$(a \rightarrow a)\$$	$(F)F'F'\$$	match-advance
$a \rightarrow a)\$$	$F)F'F'\$$	$F \rightarrow aF'$
$a \rightarrow a)\$$	$aF')F'F'\$$	match-advance
$\rightarrow a)\$$	$F')F'F'\$$	$F' \rightarrow \rightarrow FF'$
$\rightarrow a)\$$	$\rightarrow FF')F'F'\$$	match-advance
$a)\$$	$FF')F'F'\$$	$F \rightarrow aF'$
$a)\$$	$aF'F')F'F'\$$	match-advance
$)\$$	$F'F')F'F'\$$	$F' \rightarrow \epsilon$
$)\$$	$F')F'F'\$$	$F' \rightarrow \epsilon$
$)\$$	$)F'F'\$$	match-advance
$\$$	$F'F'\$$	$F' \rightarrow \epsilon$
$\$$	$F'\$$	$F' \rightarrow \epsilon$
$\$$	$\$$	分析成功

三、 (1) 语句 “ $\neg a \rightarrow a$ ” 的两棵不同的语法树为：

语法树 1:



语法树 2:



(2) 无二义文法:

$$F \rightarrow N \rightarrow F \mid N$$

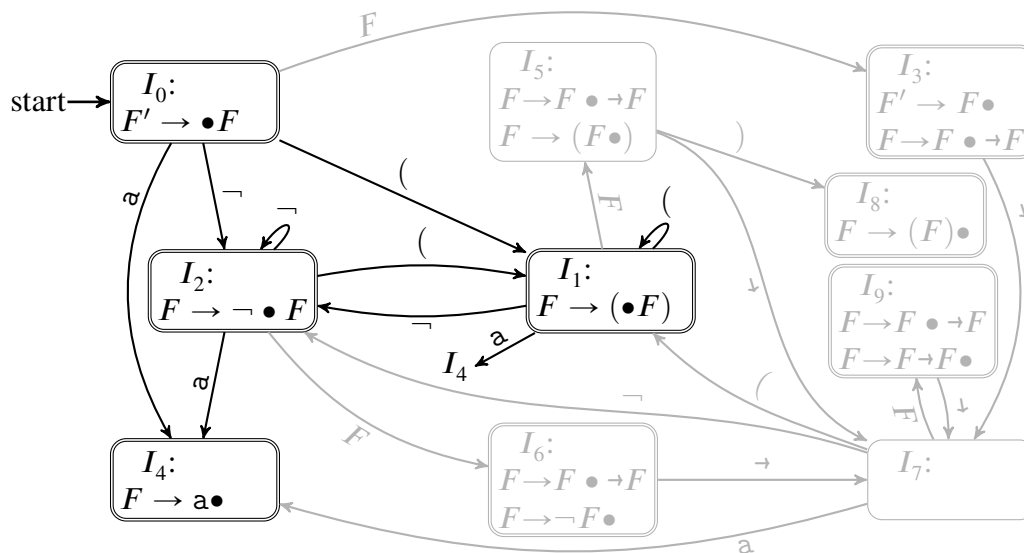
$$N \rightarrow \neg N \mid A$$

$$A \rightarrow (F) \mid a$$

四、 (1) 状态 I_7 的 LR(0) 项目集为

$$\overline{\{F \rightarrow F \rightarrow \bullet F\}} \\ = \{F \rightarrow F \rightarrow \bullet F, F \rightarrow \bullet F \rightarrow F, F \rightarrow \bullet \neg F, F \rightarrow \bullet (F), F \rightarrow \bullet a\}.$$

(2) 仅由终结符号组成的活前缀对应的前缀 DFA 为:



其对应的正则表达式为: $(\neg | ()^* a ?$.

(3) $\text{Follow}(F) = \{ \rightarrow,), \$ \}$. 状态 I_6 和状态 I_9 面对 ‘ $\rightarrow$ ’ 有移进/归约冲突. 分析表如下所示:

状态	action						goto
	a	$\neg$	$\rightarrow$	(	)	\$	F
0	s4	s2		s1			3
1	s4	s2		s1			5
2	s4	s2		s1			6
3			s7			acc	
4			r4		r4	r4	
5			s7		s8		
6			r2		r2	r2	
7	s4	s2		s1			9
8			r3		r3	r3	
9			s7		r1	r1	

(4) 语句 “ $\neg a \rightarrow a$ ” 的分析过程如下所示:

剩余串	分析栈	分析动作
$\neg a \rightarrow a \$$	0	shift
$a \rightarrow a \$$	0 $\neg$ 2	shift
$\rightarrow a \$$	0 $\neg$ 2a4	reduce $F \rightarrow a$

$\rightarrow a\$$	$0 \rightarrow 2F6$	reduce $F \rightarrow \neg F$
$\rightarrow a\$$	$0F3$	shift
$a\$$	$0F3 \rightarrow 7$	shift
$\$$	$0F3 \rightarrow 7a4$	reduce $F \rightarrow a$
$\$$	$0F3 \rightarrow 7F9$	reduce $F \rightarrow F \rightarrow F$
$\$$	$0F3$	reduce 分析成功

五、 (1)

产生式	语义规则
$F \rightarrow F_1 \rightarrow F_2$	if ($F.is_neg == True$) then $F.nnf = "(" + F_1.nnf + ")" \wedge "(" + F_2.nnf + ")"$ else $F.nnf = F_1.nnf + "\vee" + F_2.nnf$
$F \rightarrow \neg F_1$	$F.nnf = F_1.nnf$
$F \rightarrow (F_1)$	$F.nnf = F_1.nnf$
$F \rightarrow a$	if ($F.is_neg == True$) then $F.nnf = "\neg" + a.lexeme$ else $F.nnf = a.lexeme$

(2) $((A) \wedge (B) \vee C) \wedge (\neg D \vee E)$

六、

```

L0: if (g<h) goto L1      |      z := t1
    ifnot (i<j) goto L1  | L1: ifnot (a<b) goto L0
    t0 := y+1             |      ifnot (c<d) goto L2
    x := t0               |      if (e<f) goto L0
    t1 := x+y             | L2:

```

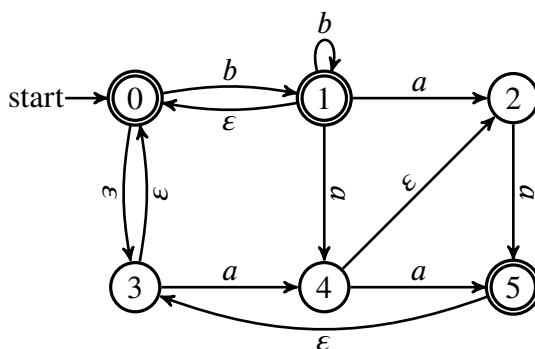
七、 `char a[][8] = "Stay", "Safe", "From", "Covid"` 把字符串保存在局部数组中, `init(s)` 初始化 `s[i]` 为指向局部数组的指针, 造成 `dangling reference`. 而 `char *a[] = "Stay", "Safe", "From", "Covid"` 初始化 `a` 为指向字符串常量的指针, 不会产生 `dangling reference`.

武汉大学计算机学院
2020 - 2021 学年第二学期 2018 级
《编译原理》期末考试试卷 (A)

学号: _____ 姓名: _____ 专业: _____ 成绩: _____

(注: ①考试时间为 120 分钟; ②所有的解答必须写在答题纸上, 并注明题号。)

一、 设 NFA N 的状态转换图如下所示: (25 分, 每小题 5 分)



- (1) 试写出 NFA N 接受字符串 “baabbbaa” 的过程;
- (2) 设用子集构造法求出的与 NFA N 等价的 DFA M 有 4 个状态 A, B, C 和 D , 其中 $A = \epsilon\text{-closure}(\{0\})$, 状态转换函数 $Dtrans(A, a) = B, Dtrans(A, b) = C$ 试求与状态 A, B, C 和 D 所对应的 NFA N 的状态集, 并画出 DFA M 的状态转换图;
- (3) 求 DFA M 的最小状态自动机;
- (4) 试用自然语言描述 NFA N 所接受的语言;
- (5) 求正则表达式 r , 使得 $L(r) = L(N)$.

二、 设嵌套序偶列表 $G(S)$ 定义如下:

$$\begin{aligned} S &\rightarrow SS \mid P \\ P &\rightarrow (E, E) \\ E &\rightarrow S \mid \text{id} \end{aligned}$$

其中: ‘id’, ‘(’, ‘)’, 和 ‘,’ 为终结符, S 是文法开始符号. (25 分, 每小题 5 分)

- (1) 试写出语句 “(id, (id, id))” 的一个最左推导;
- (2) 试消除文法 $G(S)$ 中的左递归和左公因子;
- (3) 试对消除左递归后的文法所有非终结符求 First 集和 Follow 集;
- (4) 试对消除左递归后的文法构造 LL(1) 分析表, 从而说明消除左递归后的文法不是 LL(1) 文法;
- (5) 试利用你的分析表写出语句 “(id, id)” 的一个正确的分析过程.

三、 设文法 $G(S)$ 如题二所示：

(10 分, 5+5)

- (1) 试对语句 “(id,id)(id,id)(id,id)” 画出两棵不同的语法树，从而说明该文法为二义文法；
- (2) 试设计一个与文法 $G(S)$ 等价的无二义的文法，使得序偶链接运算 (SS) 为左结合。

四、 设题二文法 $G(S)$ 的拓广文法 $G(S')$ 如下所示：

(20 分, 5+5+5+5)

$$S' \rightarrow S \quad (0)$$

$$S \rightarrow SS \quad (1)$$

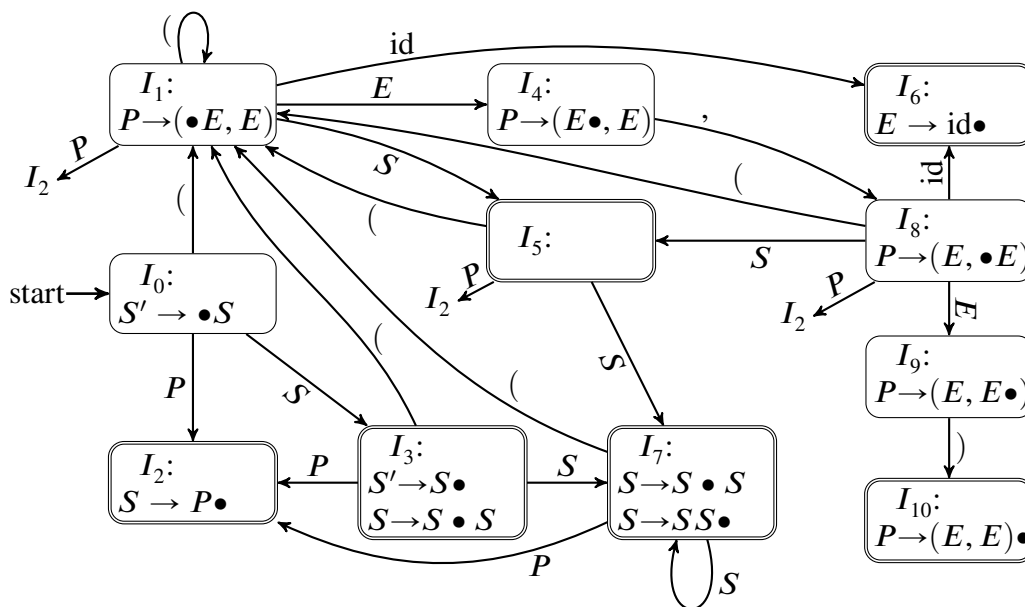
$$| P \quad (2)$$

$$P \rightarrow (E, E) \quad (3)$$

$$E \rightarrow S \quad (4)$$

$$| id \quad (5)$$

文法 $G(S')$ 的识别活前缀 LR(0) 项目自动机 M 如下图所示 (注意每个状态仅列出了核心项目，状态 I_5 除外)：



- (1) 试求状态 I_5 所对应的 LR(0) 项目集；
- (2) 试问 “(E, E,” 是活前缀吗，为什么？
- (3) 试构造该文法的 SLR 分析表，并对分析表中的移进/归约和归约/归约冲突选择正确的移进或归约动作，使得文法 $G(S)$ 的所有语句能被正确地分析且运算的优先级与结合次序与题三所规定的一致；
- (4) 试利用你的分析表写出 “(id,id,id)” 的分析过程，从而说明它不是文法 $G(S)$ 的语句。

- 五、若把题二文法 $G(S)$ 的 id 看成有向图的顶点，所生成的序偶看成有向图的边，如： $(a,b)(c,d)$ 表示有向图有四个顶点 a 、 b 、 c 和 d 及两条边 (a,b) 和 (c,d) ； $((a,b)(c,d),(e,f))$ 可把 $(a,b)(c,d)$ 和 (e,f) 看成两个子图 G_1 和 G_2 ， G_1 第一个顶点 a 到 G_2 的第一个顶点 e 有有向边。现需将文法 $G(S)$ 所生成的语句转换成绘图软件 Grapviz 输入文件格式。如

序号	嵌套序偶列表	Graphviz 输入
1	$(a,b)(c,d)$	$a \rightarrow b \quad c \rightarrow d$
2	$((a,b)(c,d),e)$	subgraph { $a \rightarrow b \quad c \rightarrow d$ } $a \rightarrow e$
3	$((a,b)(c,d),(e,f))$	subgraph { $a \rightarrow b \quad c \rightarrow d$ } subgraph { $e \rightarrow f$ } $a \rightarrow e$
4	$((((a,b)(c,d),(e,f)),g,h))$	subgraph { subgraph { $a \rightarrow b \quad c \rightarrow d$ } subgraph { $e \rightarrow f$ } $a \rightarrow e$ } subgraph { $g \rightarrow h$ } $a \rightarrow g$

为此设计为非终结符 S, E 及 P 设计综合属性 $graph$ 和 $node$ ， $graph$ 的值为该非终结符所表示的语法成分所对应的 Graphviz 语句； $node$ 的值为其表示的语法成分第一个顶点。如 S 的表示的语法成分是 $(a,b)(a,c)$ 时， $S.graph = "a \rightarrow b \quad a \rightarrow c"$ ， $S.node = "a"$ 。 E 同时有取值为布尔量的属性 is_id ，当 E 是由 id 生成时，其取值为 $true$ ，否则取值为 $false$ 。属性 $id.lexeme$ 取值为终结符 id 所对应的字符串：

- (1) 试写出文法 $G(S)$ 翻译成 Graphviz 绘图语言的语法制导定义；
- (2) 试语句 “ $((((a,b),(c,d)(e,f))(g,h),(i,m)))$ ” 转换后的 Graphviz 语句。

- 六、设有如下 Pascal 程序片段： (5 分)

```
while not(a < b and (c < d or not (e < f))) do
begin
  x := x + 1;
  if not (x = 0) and not (x = 2) then continue;
  else x := x + 2;
end;
```

其对应的三地址码如下所示

```
L1: [   ] (a < b) goto L__ | [   ] (x = 2) goto L__
      [   ] (c < d) goto L__ | L0: t1 := x + 2
      [   ] (e < f) goto L__ | x := t1
L2: t0 := x + 1 | goto L1
      x := t0 | L3:
      [   ] (x = 0) goto L__ |
```

试为其中空白 “__” 填上正确的标号编号，并为空白 “[]” 填上 if 或 $ifnot$ 。

七、 设有如下 C 语言程序:

(5 分)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void alloc(char *s)
5 {
6     s = (char *) malloc(20);
7 }
8
9 int main(void)
10 {
11     char *s;
12     alloc(s);
13     sprintf(s, "%s", "be away from covid!");
14     printf("%s\n", s);
15     return 0;
16 }
```

该程序在 Intel x86/Linux 下用 gcc 编译没有任何警告, 但运行编译后的程序报错如下:

Segmentation fault (core dumped)

试分析产生运行错误的原因.

武汉大学计算机学院 2020 - 2021 学年第二学期
2018 级《编译原理》(期末考试参考答案 A)

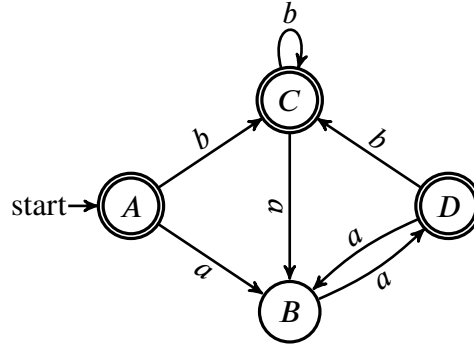
一、 (1)



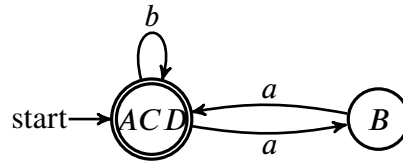
(2)

$$A = \{0, 3\}, B = \{2, 4\}, C = \{0, 1, 3\}, D = \{0, 3, 5\}.$$

状态转换图为:



(3) 最小 DFA 如下所示:



由 a 和 b 组成且 aa 以偶数的形式连续出现的字符串.

(4) $r = (b \mid aa)^*$.

二、 (1) 语句 “(id, (id, id))” 的一个最左推导如下:

$$\begin{array}{lll}
 S & \xRightarrow{lm} P & \xRightarrow{lm} (id, P) \\
 & \xRightarrow{lm} (E, E) & \xRightarrow{lm} (id, (E, E)) \\
 & \xRightarrow{lm} (id, E) & \xRightarrow{lm} (id, (id, E)) \\
 & \xRightarrow{lm} (id, S) & \xRightarrow{lm} (id, (id, id))
 \end{array}$$

(2) 消除左递归和左公因子后的文法如下:

$$\begin{array}{ll}
 S & \rightarrow PS' \\
 S' & \rightarrow SS' \mid \epsilon \\
 P & \rightarrow (E, E) \\
 E & \rightarrow S \mid id
 \end{array}$$

(3)

非终结符	First	Follow
S	(	(,), ,, \$
S'	(, ϵ	(,), ,, \$
P	(	(,), ,, \$
E	(, id	), ,

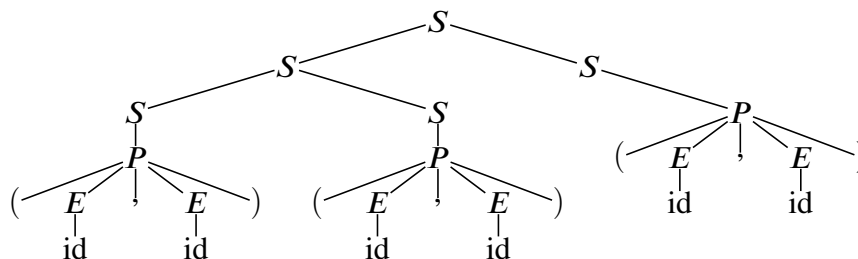
(4) LL(1) 分析表如下所示

	id	(	)	,	\$
S		$S \rightarrow PS'$			
S'		$S \rightarrow SS' \mid \epsilon$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$	$S \rightarrow \epsilon$
P		$P \rightarrow (E, E)$			
E	$E \rightarrow id$	$E \rightarrow S$			

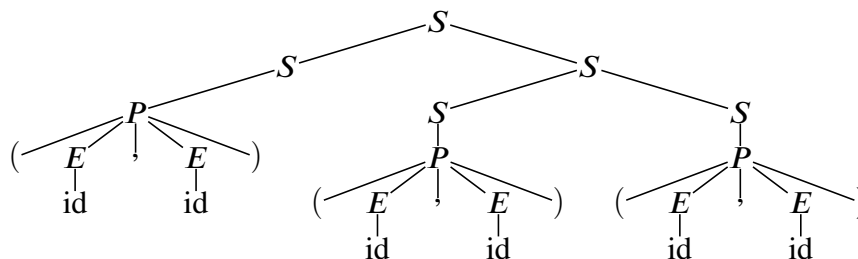
(5) 语句“(id,id)”的分析过程如下所示:

剩余串	分析栈	分析动作
(id,id)\$	$S\$$	$S \rightarrow PS'$
(id,id)\$	$PS'\$$	$P \rightarrow (E, E)$
(id,id)\$	$(E, E)S'\$$	match-advance
id,id)\$	$E, E)S'\$$	$E \rightarrow id$
id,id)\$	$id, E)S'\$$	match-advance
, id)\$	$, E)S'\$$	match-advance
id)\$	$E)S'\$$	$E \rightarrow id$
id)\$	$id)S'\$$	match-advance
)\$	$)S'\$$	match-advance
\$	$S'\$$	$S' \rightarrow \epsilon$
\$	\$	分析成功

三、 (1) 语句“(id,id)(id,id)(id,id)”的两棵不同的语法树为:
语法树 1:



语法树 2:



(2) 无二义文法:

$$S \rightarrow SP \mid P$$

$$P \rightarrow (E, E)$$

$$E \rightarrow S \mid \text{id}$$

四、 (1) 状态 I_5 的 LR(0) 项目集为

$$\begin{aligned} & \overline{\{E \rightarrow S\bullet, S \rightarrow S\bullet S\}} \\ & = \{E \rightarrow S\bullet, S \rightarrow S\bullet S, \\ & \quad S \rightarrow \bullet SS, S \rightarrow \bullet P, P \rightarrow \bullet(E, E)\} \end{aligned}$$

(2) “ $(E, E,$ ” 不是活前缀, 因为识别活前缀的 DFA 在吃进 $(E, E$ 后到达状态 I_9 , 而 I_9 没有 “,” 引出边.

(3) $\text{Follow}(S) = \text{Follow}(P) = \{ (,), ,, \$ \}$, $\text{Follow}(E) = \text{Follow}(P) = \{), ,, \}$. 状态 I 面对 ‘(’ 有移进/归约冲突. 分析表如下所示:

状态	action					goto		
	id	(	)	,	\$	S	P	E
0		s1				3	2	
1	s6	s1				5	2	4
2		r2	r2	r2	r2			
3		s1			acc	7	2	
4				s8				
5		s1	r4	r4		7	2	
6			r5	r5				
7		r1	r1	r1	r1	7	2	
8	s6	s1				5	2	9
9			s10					
10		r3	r3	r3	r3			

(4) “ $(\text{id}, \text{id}, \text{id})$ ” 的分析过程如下所示:

剩余串	分析栈	分析动作
$(\text{id}, \text{id}, \text{id})\$$	0	shift
$\text{id}, \text{id}, \text{id})$	0(1	shift
$, \text{id}, \text{id})$	0(1id6	reduce $E \rightarrow \text{id}$
$, \text{id}, \text{id})$	0(1E4	shift
$\text{id}, \text{id})$	0(1E4, 8	shift
$, \text{id})$	0(1E4, 8id6	reduce $E \rightarrow \text{id}$
$, \text{id})$	0(1E4, 8E9	error

故该串不是文法 $G(S)$ 的语句.

五、 (1)

产生式	语义规则
$S \rightarrow S_1 S_2$	$S.graph = S_1.graph + S_2.graph$ $S.node = S_1.node$
$S \rightarrow P$	$S.graph = P.graph$ $S.node = P.node$
$P \rightarrow (E_1, E_2)$	if $E_1.is_id$ then $tmp_1 = ""$ else $tmp_1 = "subgraph \{ " + E_1.graph + " \}"$ if $E_2.is_id$ then $tmp_2 = ""$ else $tmp_2 = "subgraph \{ " + E_2.graph + " \}"$ $P.graph = tmp_1 + tmp_2 + E_1.node + "->" + E_2.node$ $P.node = E_1.node$
$E \rightarrow S$	$E.graph = S.graph$ $E.node = S.node$ $E.is_id = false$
$E \rightarrow id$	$E.graph = id.lexeme$ $E.node = id.lexeme$ $E.is_id = true$

(2)

```

subgraph {
    subgraph {
        a -> b
    }
    subgraph {
        c -> d
        e -> f
    }
    a -> c
    g -> h
}
subgraph {
    i -> m
}
a -> i

```

六、

```

L1: ifnot (a < b) goto L2 |      ifnot (x = 2) goto L1
    if (c < d) goto L3      | L0: t1 := x + 2
    ifnot (e < f) goto L3 |      x := t1

```

```
L2: t0 := x + 1          |      goto L1
    x := t0              | L3:
    if (x = 0) goto L0
```

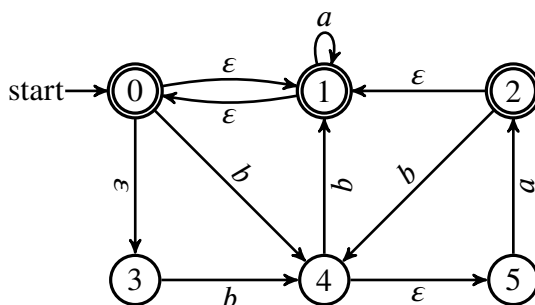
七、 由于 C 语言采用的是传值的函数调用方式，`alloc(s)` 分配的存储空间未保存在 `s` 上, 而是实参对应的临时变量上.

武汉大学计算机学院
2019 - 2020 学年第一学期 2017 级
《编译原理》期末考试试卷 (A)

学号: _____ 姓名: _____ 专业: _____ 成绩: _____

(注: ①考试时间为 120 分钟; ②所有的解答必须写在答题纸上, 并注明题号。)

一、 设 NFA N 的状态转换图如下所示: (25 分, 每小题 5 分)



- (1) 试写出 NFA N 接受字符串 “abaabb” 的一个过程;
- (2) 设用子集构造法求出的与 NFA N 等价的 DFA M 有 3 个状态 A , B 和 C , 其中 $A = \epsilon\text{-closure}(\{0\})$, $Dtrans(A, b) = B$. 试求与状态 A , B 和 C 所对应的 NFA N 的状态集, 并画出 DFA M 的状态转换图;
- (3) 求 DFA M 的最小状态自动机;
- (4) 试用自然语言描述 NFA N 所接受的语言;
- (5) 求正规表达式 r , 使得 $L(r) = L(N)$.

二、 设函数调用表达式文法 $G(E)$ 定义如下:

$$\begin{aligned} E &\rightarrow E(A) \mid (E) \mid \text{id} \\ A &\rightarrow A, A \mid E \end{aligned}$$

其中: ‘id’, ‘,’ , ‘(’ 和 ‘)’ 为终结符, E 是文法开始符号. (25 分, 每小题 5 分)

- (1) 试写出语句 “id(id, id)” 的一个最左推导;
- (2) 试消除文法 $G(E)$ 中的左递归;
- (3) 试对消除左递归后的文法所有非终结符求 First 集和 Follow 集;
- (4) 试对消除左递归后的文法构造 LL(1) 分析表, 从而说明消除左递归后的文法不是 LL(1) 文法;
- (5) 试利用你的分析表写出语句 “id(id)” 的一个正确的分析过程.

(10 分, 5+5)

- (1) 试对语句 “ $\text{id}(\text{id}, \text{id}, \text{id})$ ” 画出两棵不同的语法树, 从而说明该文法为二义文法;
- (2) 试设计一个与文法 $G(E)$ 等价的无二义的文法, 使得参数列表 (A, A) 为左结合.

(20 分, 5+5+5+5)

$$E' \rightarrow E \quad (0)$$

$$E \rightarrow E(A) \quad (1)$$

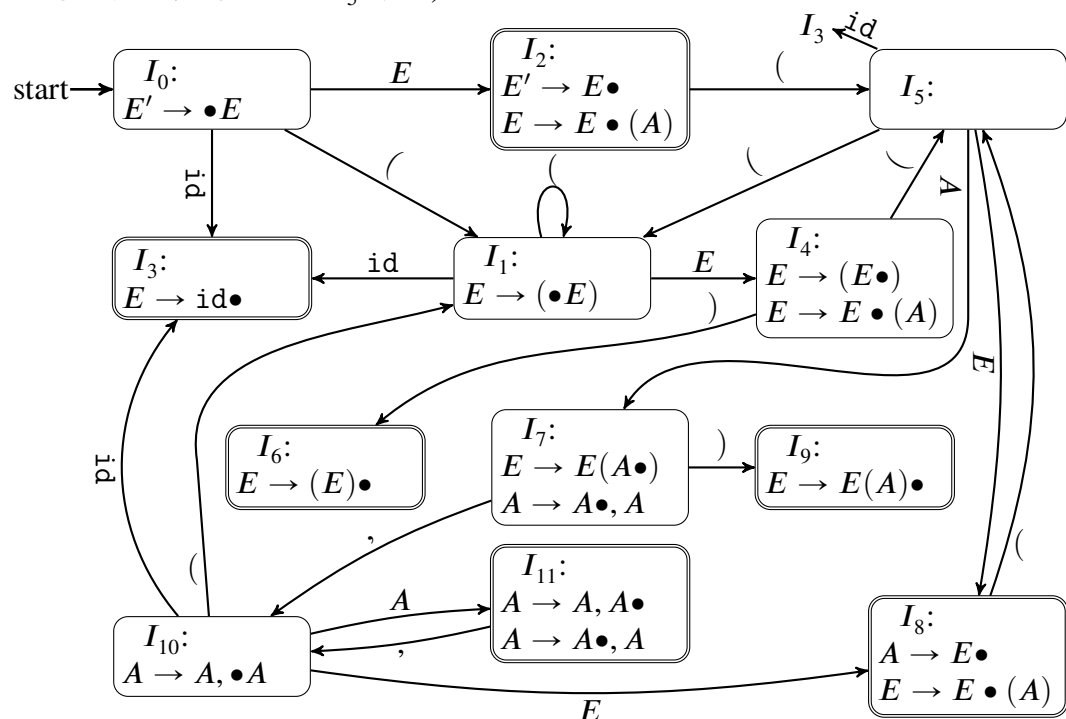
$$| \quad (E) \quad (2)$$

$$\text{id} \quad (3)$$

$$A \rightarrow A, A \quad (4)$$

$$|E| \tag{5}$$

文法 $G(E')$ 的识别活前缀 LR(0) 项目自动机 M 如下图所示 (注意每个状态仅列出了核心项目, 状态 I_5 除外):



- (1) 试求状态 I_5 所对应的 LR(0) 项目集;
- (2) 试求终结符号串集合 $\{w \mid E(\underline{A}, w \in L(M) \wedge w \in \{\text{id}, ,, (,)\}^*)\}$ (即活前缀 “ $E(\underline{A},$ ” 加上后缀 w 能形成句柄) 所对应的正则表达式;
- (3) 试构造该文法的 SLR 分析表, 并对分析表中的移进/归约冲突选择正确的移进或归约动作, 使得文法 $G(E)$ 的所有语句能被正确地分析且运算的结合次序与题三所规定的一致;
- (4) 试利用你的分析表写出语句 “id(id)” 的分析过程.

五、现需对题二文法 $G(E)$ 所生成的函数调用表达式柯里化 (currying)，即所有的函数只允许有一个参数，且函数调用 $f(x)$ 记为 $f\ x$ 。若 f 是二元函数，柯里化把 f 看成高阶函数，即 $f\ x$ 的返回值是函数，这样 $f(x,y)$ 的柯里化为 $(f\ x)\ y$ ，简记为 $f\ x\ y$ (柯里化的函数调用为左结合)。函数调用表达式的最简柯里化 (括号最少) 用例如下： (10 分, 5+5)

序号	表达式	最简柯里化
1	$f(a,b,c,d)$	$f\ a\ b\ c\ d$
2	$(f(a,b))(c,d)$	$f\ a\ b\ c\ d$
3	$((f(a))(b))(c,d)$	$f\ a\ b\ c\ d$
4	$(f(a(b)))(c,d)$	$f\ (a\ b)\ c\ d$
5	$f(a(c,d),e(g,h))$	$f\ (a\ c\ d)\ (e\ g\ h)$

为此设计以下综合属性： $E.kind$ 和 $A.kind$ ，其取值为选项 OPEN 或 CLOSE。当 E 所表示的语法成分是 id 或 $E(A)$ 时， $E.kind$ 的取值分别是 CLOSE 或 OPEN。当 A 所表示的语法成分是 A, A 或 E 时， $A.kind$ 的取值分别是 CLOSE 或 $E.kind$ ； $E.curry$ 和 $A.curry$ 取值为其表示的语法成分最简柯里化表达式 (字符串)；终结符 id 有属性 $id.lexeme$ ，其取值为 id 所对应的字符串。

- (1) 试设计最简柯里化 SDD；
- (2) 试求 “ $(a(b(c(d,e),f),g))(h(i,j))$ ” 的最简柯里化。

六、设有如下 Pascal 程序片段： (5 分)

```
while a > b and not (c > d) do
begin
  x := x + 1;
  if e > f and g > h or not (i > k) then continue;
  else x := y + 2;
end;
```

其对应的三地址码如下所示

```
L1: [ ] (a > b) goto L__ | L0: [ ] (i > k) goto L__
    [ ] (c > d) goto L__ |      t0 := y + 2
    x := x + 1           |      x := t0
    [ ] (e > f) goto L__ |      goto L__
    [ ] (g > h) goto L__ | L2:
```

试为其中空白 “__” 填上正确的标号编号，并为空白 “[]” 填上 if 或 ifnot。

第七题见下页！

七、 设有如下 C 语言程序:

(5 分)

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 char **foo(char **src, int n)
6 {
7     int i = 0;
8     char **dest;
9     dest = (char **) malloc(sizeof(char *) * n);
10    while(i < n) {
11        dest[i] = strdup(src[i]);
12        i++;
13    }
14    return dest;
15 }
16
17 int main()
18 {
19     char s[][20] = {"Hello", "Class 2017"};
20     char **dest;
21     dest = foo((char **)s, 2);
22     for(int i = 0; i < 2; i++)
23         printf("%s\n", dest[i]);
24     return 0;
25 }
```

该程序希望拷贝一个字符串数组, 并输出拷贝后的数组. 在 Intel x86/Linux 下用 gcc 编译没有任何警告, 但运行编译后的程序报 “Segmentation fault”. 但若把程序的第 19 行修改为:

```
19 char *s[] = {"Hello", "Class 2017"};
```

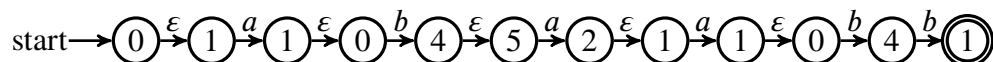
则程序工作正常, 即在输出以下字符串后正常退出:

```
Hello
Class 2017
```

试问上述程序产生运行错误的原因.

武汉大学计算机学院 2019 - 2020 学年第一学期
2017 级《编译原理》(期末考试参考答案 A)

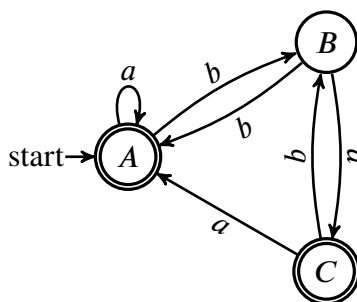
一、 (1)



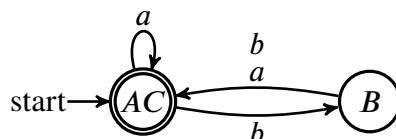
(2)

$$A = \{0, 1, 3\}, B = \{4, 5\}, C = \{0, 1, 2, 3\},$$

状态转换图为:



(3) 最小 DFA 如下所示:



(4) 不以奇数 b 结尾.

(5) $r = (bb \mid ba \mid a)^*$.

二、 (1) 语句 “id(id, id)” 的最左推导如下:

$$\begin{array}{lll} E & \xRightarrow{lm} E(A) & \xRightarrow{lm} id(E, A) \\ & \xRightarrow{lm} id(A) & \xRightarrow{lm} id(id, A) \\ & \xRightarrow{lm} id(A, A) & \xRightarrow{lm} id(id, id) \end{array}$$

(2) 消除左递归后的文法如下:

$$\begin{array}{l} E \rightarrow (E)E' \mid id E' \\ E' \rightarrow (A)E' \mid \epsilon \\ A \rightarrow E A' \\ A' \rightarrow , A A' \mid \epsilon \end{array}$$

(3)

非终结符	First	Follow
E	id (	), \$
E'	ϵ (	), \$
A	id (	),
A'	ϵ ,	),

(4) LL(1) 分析表如下所示

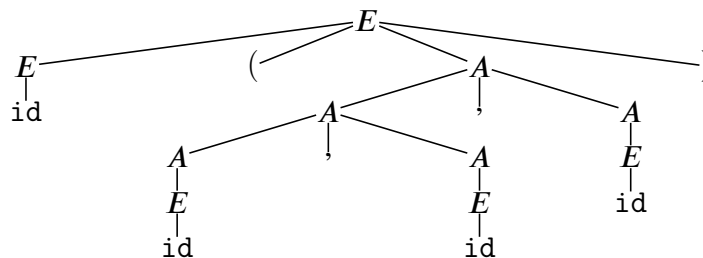
	id	(	)	,	\$
E	$E \rightarrow idE'$	$E \rightarrow (E)E'$			
E'		$E' \rightarrow (A)E'$	$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
A	$A \rightarrow EA'$	$A \rightarrow EA'$			
A'			$A \rightarrow \epsilon$	$A' \rightarrow \epsilon \mid ,AA'$	

(5) 语句 “id(id)” 的分析过程如下所示：

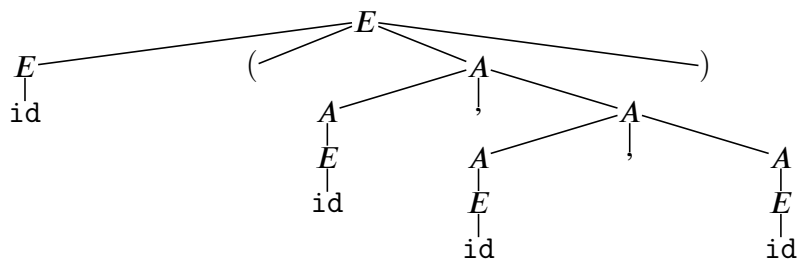
剩余串	分析栈	分析动作
id(id)\$	$E\$$	$E \rightarrow idE'$
id(id)\$	$idE'\$$	match-advance
(id)\$	$E'\$$	$E' \rightarrow (A)E'$
(id)\$	$(A)E'\$$	match-advance
id)\$	$A)E'\$$	$A \rightarrow EA'$
id)\$	$EA')E'\$$	$E \rightarrow idE'$
id)\$	$idE'A')E'\$$	match-advance
)\$	$E'A')E'\$$	$E' \rightarrow \epsilon$
)\$	$A')E'\$$	$A' \rightarrow \epsilon$
)\$	$)E'\$$	match-advance
\$	$)E'\$$	$E' \rightarrow \epsilon$
\$	\$	分析成功

三、 (1) 语句 “id(id,id,id)” 的两棵不同的语法树为：

语法树 1:



语法树 2:



(2) 无二义文法:

$$\begin{aligned}
 E &\rightarrow E(A) \mid (E) \mid id \\
 A &\rightarrow A, E \mid E
 \end{aligned}$$

四、 (1) 状态 I_5 的 LR(0) 项目集为

$$\overline{\{E \rightarrow E(\bullet A)\}} \\ = \{E \rightarrow E(\bullet A), A \rightarrow \bullet A, A, A \rightarrow \bullet E, E \rightarrow \bullet(E), E \rightarrow \bullet E(A), E \rightarrow \bullet id\}.$$

(2) 识别活前缀的自动在吃进 $E(A$, 进入状态 I_{10} , 状态 I_{10} 还能接受形成句柄的终结符号串为: $(^*id$.

(3) $\text{Follow}(E) = \{ (,), ,, \$ \}$, $\text{Follow}(A) = \{), ,, \}$. 状态 I_{11} 面对 “,” 有移进/归约冲突, 左结合选归约. 分析表如下所示:

状态	action					goto	
	id	(	)	,	\$	E	A
0	s3	s1				2	
1	s3	s1				4	
2		s5			acc		
3		r3	r3	r3	r3		
4		s5	s6				
5	s3	s1				8	7
6		r2	r2	r2	r2		
7			s9	s10			
8		s5	r5	r5			
9		r1	r1	r1	r1		
10	s3	s1				8	11
11			r4	r4			

(4) 语句 “id(id)” 的分析过程如下所示:

剩余串	分析栈	分析动作
id(id)\$	0	shift
(id)\$	0id3	reduce $E \rightarrow id$
(id)\$	0E2	shift
id)\$	0E2(5	shift
)\$	0E2(5id3	reduce $E \rightarrow id$
)\$	0E2(5E8	reduce $A \rightarrow E$
)\$	0E2(5A7	shift
\$	0E2(5A7)9	reduce $E \rightarrow E(A)$
\$	0E2	reduce 分析成功

五、 (1)

产生式	语义规则
$E \rightarrow E_1(A)$	$E.kind = OPEN$ $lp = " "; rp = ""$ if ($A.kind == OPEN$) then $lp = " (" ; rp = ") "$ $E.curry = E_1.curry + lp + A.curry + rp$
$E \rightarrow (E_1)$	$E.kind = E_1.kind$ $E.curry = E_1.curry$
$E \rightarrow id$	$E.kind = CLOSE$ $E.curry = id.lexeme$
$A \rightarrow A_1, A_2$	$A.kind = CLOSE$ $lp1 = "" ; rp1 = " "$ $lp2 = "" ; rp2 = ""$ if ($A_1.kind == OPEN$) then $lp1 = " (" ; rp1 = ") "$ if ($A_2.kind == OPEN$) then $lp2 = " (" ; rp2 = ") "$ $A.curry = lp1 + A_1.curry + rp1 + lp2 + A_2.curry + rp2$
$A \rightarrow E$	$A.kind = E.kind$ $A.curry = E.curry$

(2) a (b (c d e) f) g (h i j)

六、

```

L1: ifnot (a > b) goto L2 | L0: ifnot (i > k) goto L1
    if (c > d) goto L2    |      t0 := y + 2
    x := x + 1           |      x := t0
    ifnot (e > f) goto L0 |      goto L1
    if (g > h) goto L1    | L2:

```

七、 s in the expression is in fact of the type pointer to array. so the value of *s is the address of the first element of the array. but if casting s as ((char **)), *s will take the value of the address of the first element of the array ('H'). so after applying second dereferencing (**s), it will cause memory access violation. if we change s to array of pointer (char *s[]), it will work correctly.

simply: diff of pointer to array and array of pointer.

武汉大学计算机学院
2023 -2024 学年度第一学期 21 级软件工程专业
《编译技术及应用》期末考试试卷 A 卷

专业：_____ 学号：_____ 姓名：_____

说明：请将答案一律写在答题纸上，注意标明题号并按顺序解答。答在试卷上不得分。
未经主考教师同意，考试试卷、答题纸、草稿纸均不得带离考场，否则视为违规。

题号	一	二	三	四	五	六	总分
总分	15	25	20	20	15	5	100

一、（15 分）设有 $\Sigma=\{0,1\}$ 上的正规集 L_1 ，包含所有“倒数第二个字符为 1”的串。

- （1）请写出正规表达式 e ，使得 $L(e)=L_1$ ；
- （2）请根据构造算法，构造出识别 L_1 的非确定的有穷状态自动机 N ，并画出其状态转换图；
- （3）请根据转换算法，将非确定的有穷状态自动机 N ，转换成状态数目最少的确定的有穷状态自动机 M ，使得 $L(M)=L(N)$ ，并画出 M 的状态转换图。

二、（25 分）设有 $\Sigma=\{a,b,c\}$ 上的上下文无关语言 L_2 ，

$$L_2 = \{\omega c^{2n+1} \omega^{-1} \mid \omega \in (a|b)^*, n \geq 1\}, \quad \text{其中, } \omega^{-1} \text{ 表示符号串 } \omega \text{ 的逆(倒置)}。$$

- （1）请用自然语言描述该语言 L_2 ；
- （2）试构造一个 LL(1)文法 $G_2(S)$ ，使得 $L(G_2)=L_2$ 。请给出文法 G_2 的形式化定义；
- （3）试给出句子 $abcccba$ 的最左推导序列；
- （4）构造文法 $G_2(S)$ 的 LL(1)分析表，证明该文法是 LL(1)文法；
- （5）给出句子 $abcccba$ 的 LL(1)语法分析过程。

三、（20 分）现有 C 语言布尔表达式文法 $G_3(B)$ ，

$$G_3(B) = (\{B\}, \{i, ||, \&\&, !, (,)\}, P, B), \quad \text{其中 } P \text{ 定义如下:}$$

$$B \rightarrow B||B$$

$$B \rightarrow B\&\&B$$

$$B \rightarrow !B$$

$$B \rightarrow (B)$$

$$B \rightarrow i$$

- （1）试用直接改写法消除文法 $G_3(B)$ 的左递归，写出该等价文法；
- （2）请画出某句子的语法分析树，证明文法 $G_3(B)$ 是二义性文法；
- （3）试设计一个等价的无二义性文法，使得运算的优先级由高到低为：括号，“!”、“&&”和“||”。其中，“!”运算右结合，其余运算均为左结合；
- （4）根据题（3）的规则，请写出句子“ $i||i\&\&!i||i$ ”的逆波兰表示。

四、（20 分）设题三文法 $G_3(B)$ 的拓广文法 $G_3(B')$ 如下所示：

$$B' \rightarrow B \quad (0)$$

$$B \rightarrow B||B \quad (1)$$

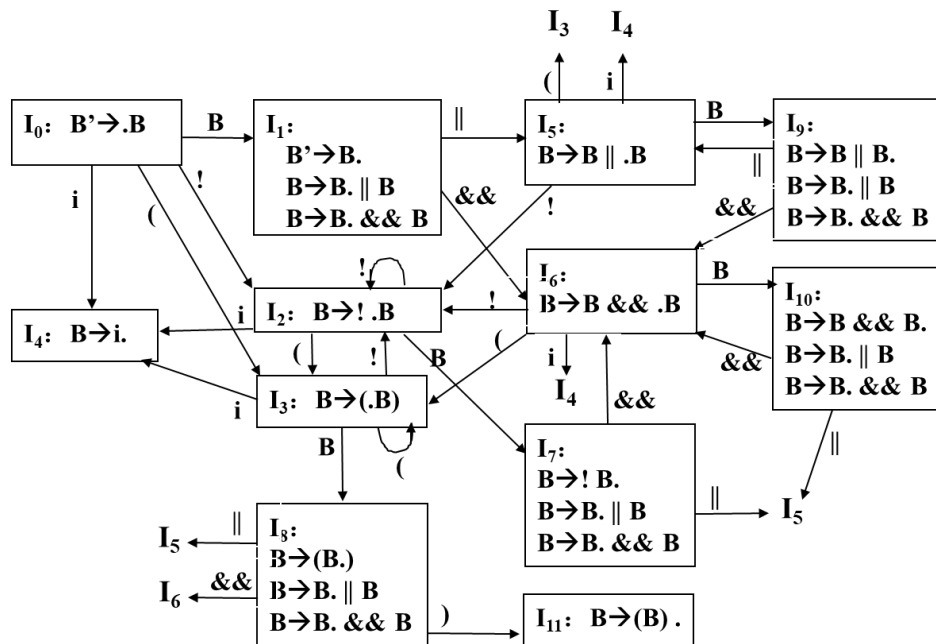
$$B \rightarrow B\&\&B \quad (2)$$

$$B \rightarrow !B \quad (3)$$

$B \rightarrow (B)$ (4)

$B \rightarrow i$ (5)

文法 $G_3(B')$ 的识别规范句型活前缀的 LR(0) 自动机 M 如下图所示（注意每个状态仅列出了核心项目）。



- 试问正规表达式 $B\|B\&\&!!^*$ 所生成的符号串是否为文法的活前缀（可行前缀）？如果是，请指出该活前缀对应的有效项目集，并写出其包含的所有 LR(0) 项目；
- 由于该文法是二义性文法，因此自动机 M 中一定存在冲突状态。请指出所有的冲突状态，并具体说明它（们）在何种情况下，分别含有怎样的冲突；
- 若运算的优先级和结合次序与题三（3）一致。请按照规定消解冲突，将文法 $G_3(B')$ 的 SLR(1) 分析表（如下表所示）补充完整（答题纸重画）；

	ACTION							GOTO
	i	$\ $	$\&\&$	!	(	)	#	
0	S ₄			S ₂	S ₃			1
1								
2								
3								
4								
5								
6								
7								
8								
9								
10								
11								

- (4) 请根据上述运算规则，按照下表格式，给出串“i||i&&! i||i”正确的 SLR(1)语法分析过程。

步骤	分析栈	余留符号串	语法分析动作
1	#0	i i&&! i i#	S ₄
⋮	⋮	⋮	⋮

五、(15 分) 假设在某语言的语法定义中，除了有 C 语言中的布尔表达式的语法定义外，还有直接定义“或非”运算布尔表达式的语法规则，产生式为 $B \rightarrow B_1 \downarrow B_2$ ，其中，“ $\downarrow$ ”表示或非运算符。

或非运算规则为：先“或”后“非”，即将运算量 B_1 和 B_2 先进行“逻辑或运算”，将结果再进行“逻辑非运算”，产生最终结果 B 。(全零为一，有一为零)。

- (1) 如果为与运算布尔表达式 $B_1 \&\& B_2$ 生成三地址代码的相关语义规则被定义为：

产生式	语义规则
$B \rightarrow B_1 \&\& B_2$	$B_1.true = newlabel()$ $B_1.false = B.false$ $B_2.true = B.true$ $B_2.false = B.false$ $B.code = B_1.code \parallel gen(B_1.true ":") \parallel B_2.code$
$B \rightarrow E_1 \text{ rel } E_2$	$B.code = E_1.code \parallel E_2.code$ $\parallel gen('if\ E_1.place\ relop.\ op\ E_2.place\ 'goto\ B.true)$ $\parallel gen('goto\ B.false)$

请按照这种翻译模式，直接给出对应于产生式 $B \rightarrow B_1 \downarrow B_2$ 的语义规则。

产生式	语义规则
$B \rightarrow B_1 \downarrow B_2$	⋮

- (2) 如果采用地址回填技术，与运算布尔表达式 $B_1 \&\& B_2$ 的三地址代码翻译方案被定义为：

产生式	语义规则
$B \rightarrow B_1 \&\& MB_2$	$\{ \text{Backpatch}(B_1.truelist, M.quad);$ $B.truelist = B_2.truelist;$ $B.falselist = merge(B_1.falselist, B_2.falselist); \}$
$M \rightarrow \varepsilon$	$\{ M.quad = nextquad; \}$
$B \rightarrow E_1 \text{ rel } E_2$	$\{ B.truelist = makelist(nextquad);$ $B.falselist = makelist(nextquad + 1);$ $gen('if\ E_1.place\ rel.op\ E_2.place\ 'goto\ _');$ $gen('goto\ _'); \}$

请按照这种翻译模式，直接给出对应于产生式 $B \rightarrow B_1 \downarrow B_2$ 的翻译方案。

产生式	语义规则
$B \rightarrow B_1 \downarrow MB_2$	⋮

- (3) 按照第(2)题的翻译方案, 假设从 100 开始给三地址代码指令编址, 请给出布尔表达式 $x < 10 \downarrow y > 20$ 的注释语法分析树, 并在这棵树上, 标注有关节点的 *truelist* 属性值和 *flaselist* 属性值。

六、(5 分) 假设有如下 C 语言程序片段,

```
while ( !(a<b) || c>d )
{
    if (e<f)
        continue;
    m =m + n;
}
```

如果与其对应的中间代码如下所示:

```
L0: [    ] (a < b) goto L__
    [    ] (c > d) goto L__
L1: [    ] (e < f) goto L__
    t0 :=m+n
    m :=t0
    goto L__
L2:
```

请在其中空白 “__” 填上正确的标号编号, 并在空白 “[]” 处填上 if 或 ifnot。

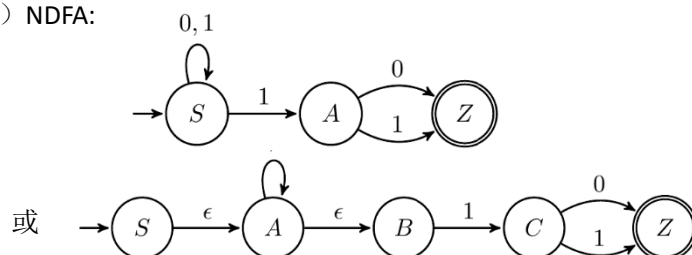
武汉大学计算机学院
2023 -2024 学年度第一学期 软件工程专业
《编译技术及应用》期末考试 A 卷 参考答案

题号	一	二	三	四	五	六	总分
总分	15	25	20	20	15	5	100

一、(15 分，每小题 5 分)

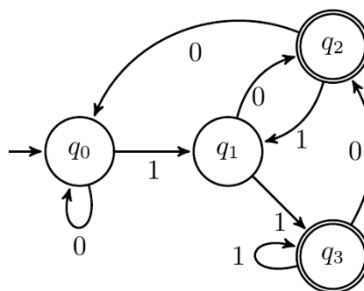
(1) $e = (0|1)^*1(0|1)$

(2) NDFA:



或其它等价的 FA。

(3) MFA:



二、(25 分，每小题 5 分)

(1) 以奇数个 c 为对称轴的 a、b 串回文结构。

(2) 文法 $G_2[S] = (\{S, C\}, \{a, b, c\}, P, S)$ ，其中，

$S \rightarrow aSa \mid bSb \mid cC$

$C \rightarrow ccC \mid \varepsilon$

(3) 句子 abcccba 的最左推导： $S \xRightarrow{lm} aSa \xRightarrow{lm} abSba \xRightarrow{lm} abcCba \xRightarrow{lm} abcccCba \xRightarrow{lm} abcccba$

(4) 构造文法 G_2 的 LL(1)分析表如下。

	a	b	c	\$
S	$S \rightarrow aSa$	$S \rightarrow bSb$	$S \rightarrow cC$	
C	$C \rightarrow \varepsilon$	$C \rightarrow \varepsilon$	$C \rightarrow ccC$	$C \rightarrow \varepsilon$

表中不存在多重定义的（冲突）表项，因此， G_2 是 LL(1)文法。

(5) 句子 abcccba 的 LL(1)语法分析过程如下：

符号栈	余留符号串	分析动作
S#	abcccba#	$S \rightarrow aSa$
aSa#	abcccba#	'a' matched
Sa#	bcccba#	$S \rightarrow bSb$
bSba#	bcccba#	'b' matched
Sba#	cccba#	$S \rightarrow cC$
cCba#	cccba#	'c' matched

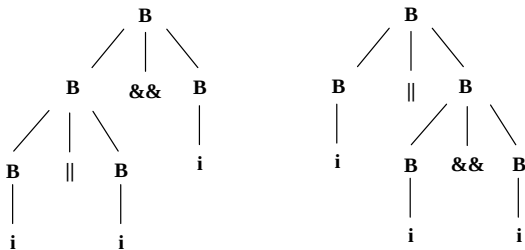
Cba#	ccba#	$C \rightarrow ccC$
ccCba#	ccba#	'c' matched
cCba#	cba#	'c' matched
Cba#	ba#	$C \rightarrow \varepsilon$
ba#	ba#	'b' matched
a#	a#	'a' matched
#	#	OK

三、(20 分，每小题 5 分)

$$(1) \quad B \rightarrow !BB' \mid (B)B' \mid iB'$$

$$B' \rightarrow \parallel BB' \mid \&\&BB' \mid \varepsilon$$

(2) 句子 “i||i&i” 有两棵语法分析树，因此，该文法是二义性文法。



$$(3) \quad B \rightarrow B \parallel T \mid T$$

$$T \rightarrow T \&\&F \mid F$$

$$F \rightarrow !F \mid P$$

$$P \rightarrow (B) \mid i$$

(4) 逆波兰式为 $iii! \&\& \parallel i \parallel$

四、(20 分，每小题 5 分)

- (1) 是，对应的有效项目集为 I_2 。
 $I_2 = \{[B \rightarrow !.B], [B \rightarrow .B \parallel B], [B \rightarrow .B \&\&B], [B \rightarrow .!B], [B \rightarrow .(B)], [B \rightarrow .i]\}$ 。
- (2) I_7 、 I_9 和 I_{10} 三个状态面对 “||” 和 “&&” 符号时都有移进/归约冲突。
- (3) 根据语义规则，消解冲突后的 SLR(1)分析表：

	ACTION							GOTO
	i		&&	!	(	)	#	
0	S4			S2	S3			1
1		S5	S6				acc	
2	S4			S2	S3			7
3	S4			S2	S3			8
4		r5	r5			r5	r5	
5	S4			S2	S3			9

6	S4			S2	S3			10
7		r3 /s5	r3 /s6			r3	r3	
8		S5	S6			S11		
9		r1 /s5	S6 /s6			r1	r1	
10		r2 /s5	r2 /s6			r2	r2	
11		r4	r4			r4	r4	

(4) 串“i || i && ! i || i”正确的 SLR(1)语法分析过程

步骤	栈	输入	语法分析动作
1	#0	i i&&!i i#	s4
2	#0i4	i&&!i i#	r5:B→i
3	#4B1	i&&!i i#	s5
4	#0B1 5	i&&!i i#	s4
5	#0B1 5i4	&&!i i#	r5:B→i
6	#0B1 5B9	&&!i i#	S6: &&优先于
7	#0B1 5B9&&6	!i i#	s2
8	#0B1 5B9&&6!2	i i#	s4
9	#0B1 5B9&&6!2i4	i#	r5: B→i
10	#0B1 5B9&&6!2B7	i#	r3 B → ! B !优先于
11	#0B1 5B9&&6B10	i#	r2 B → B&&B &&优先于
12	#0B1 5B9	i#	r1 B → B B 左结合
13	#0B1	i#	s5
14	#0B1 5	i#	s4
15	#0B1 5i4	#	r5: B→i
16	#0B1 5B9	#	r1: B → B B
17	#0B1	#	acc!

五、(15 分，每小题 5 分)

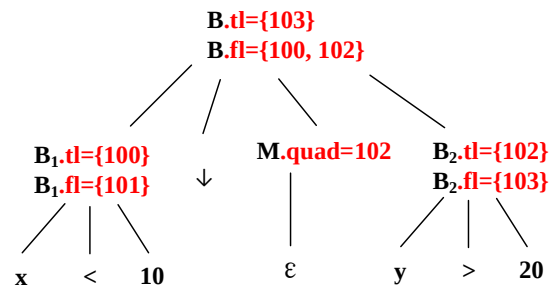
(1) 多趟扫描：

产生式	语义规则
$B \rightarrow B_1 \downarrow B_2$	$B_1.true = B.false$ $B_1.false = newlabel()$ $B_2.true = B.false$ $B_2.false = B.true$ $B.code = B_1.code label(B_1.false) B_2.code$

(2) 单趟扫描地址回填：

产生式	翻译方案
$B \rightarrow B_1 \downarrow MB_2$	$\{ Backpatch(B_1.falselist, M.quad)$ $B.truelist = B_2.falselist$ $B.falselist = merge(B_1.truelist, B_2.truelist); \}$

(3) 附注语法树（注释树）：



六、(5 分)

```

L0:  [ ifnot ] (a < b) goto L_1
      [ ifnot ] (c > d) goto L_2
L1:  [   if   ] (e < f) goto L_0
      t0 :=m+n
      m :=t0
      goto L_0
L2:
  
```

Assignment 1

1.1 Consider the context-free grammar:

$s \rightarrow s s + \mid s s * \mid a$ (2.2.1)

1. Show how the string $aa + a^*$ can be generated by this grammar.
2. Construct a parse tree for this string.
3. What language does this grammar generate?

1.2 Construct unambiguous context-free grammars for each of the following languages. (2.2.4)

1. Arithmetic expressions in postfix notation.
2. Right-associative lists of identifiers separated by commas.
3. Arithmetic expressions of integers and identifiers with the four binary operators $+$, $-$, $*$, $/$.
4. Add unary plus and minus to the arithmetic operators of 4.

1.3 Write regular definitions for the following languages: (3.3.5)

1. All strings of lowercase letters that contain the five vowels in order.
2. All strings of lowercase letters in which the letters are in ascending lexicographic order.
3. All strings of digits with no repeated digits. Hint: Try this problem first with a few digits, such as $\{0, 1, 2\}$.
4. All strings of digits with at most one repeated digit.
5. All strings of a's and b's with an even number of a's and an odd number of b's.
6. All strings of a's and b's that do not contain the substring abb.
7. All strings of a's and b's that do not contain the subsequence abb.

1.4 Convert the following regular expressions to DFA: (3.7.3)

1. $(a|b)^*$
2. $(a^*|b^*)^*$
3. $((\epsilon|a)|b^*)^*$

Assignment 1

1.1 Consider the context-free grammar:

$S \rightarrow S S + \mid S S * \mid a$ (2.2.1)

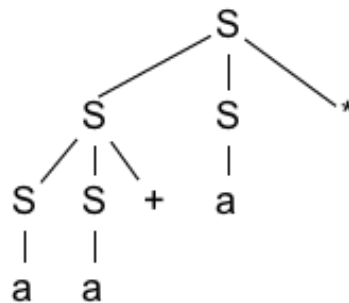
1. Show how the string `aa + a*` can be generated by this grammar.
2. Construct a parse tree for this string.
3. What language does this grammar generate?

Answer:

1.

$S \rightarrow S S * \rightarrow S S + S * \rightarrow a S + S * \rightarrow a a + S * \rightarrow a a + a *$

2.



3. $L = \{\text{Postfix expression consisting of digits, plus and multiple signs}\}$

1.2 Construct unambiguous context-free grammars for each of the following languages. (2.2.4)

1. Arithmetic expressions in postfix notation.
2. Right-associative lists of identifiers separated by commas.
3. Arithmetic expressions of integers and identifiers with the four binary operators $+$, $-$, $*$, $/$.
4. Add unary plus and minus to the arithmetic operators of 4.

Answer:

```

1. E -> E E op | num

2. list -> id , list | id

3. expr -> expr + term | expr - term | term
   term -> term * factor | term / factor | factor
   factor -> id | num | (expr)

4. expr -> expr + term | expr - term | term
   term -> term * unary | term / unary | unary
   unary -> + factor | - factor | factor
   factor -> id | num | (expr)

```

1.3 Write regular definitions for the following languages: (3.3.5)

1. All strings of lowercase letters that contain the five vowels in order.
2. All strings of lowercase letters in which the letters are in ascending lexicographic order.
3. All strings of digits with no repeated digits. Hint: Try this problem first with a few digits, such as {0, 1, 2}.
4. All strings of digits with at most one repeated digit.
5. All strings of a's and b's with an even number of a's and an odd number of b's.
6. All strings of a's and b's that do not contain the substring abb.
7. All strings of a's and b's that do not contain the subsequence abb.

Answer:

```

(!!The answer is not unique)
1. want -> other* a (other|a)* e (other|e)* i (other|i)* o (other|o)* u (other|u)*
   other -> [bcdfghjklmnpqrstvwxyz]

2. a* b* ... z*

3. want -> 0|A?0?1(A0?1|01)*A?0?|A0?
   A -> 0?2(02)*

4. 略 (思路在文件最后)

5. want -> (FE*G|(aa)*b)(E|FE*G)
   E -> b(aa)*b
   F -> a(aa)*b
   G -> b(aa)*ab|a
   F -> ba(aa)*b

6. b*(a+b?)*

7. b* | b*a+ | b*a+ba*

```

1.4 Convert the following regular expressions to DFA: (3.7.3)

- 1. $(a|b)^*$
- 2. $(a^*|b^*)^*$
- 3. $((\epsilon|a)b^*)^*$

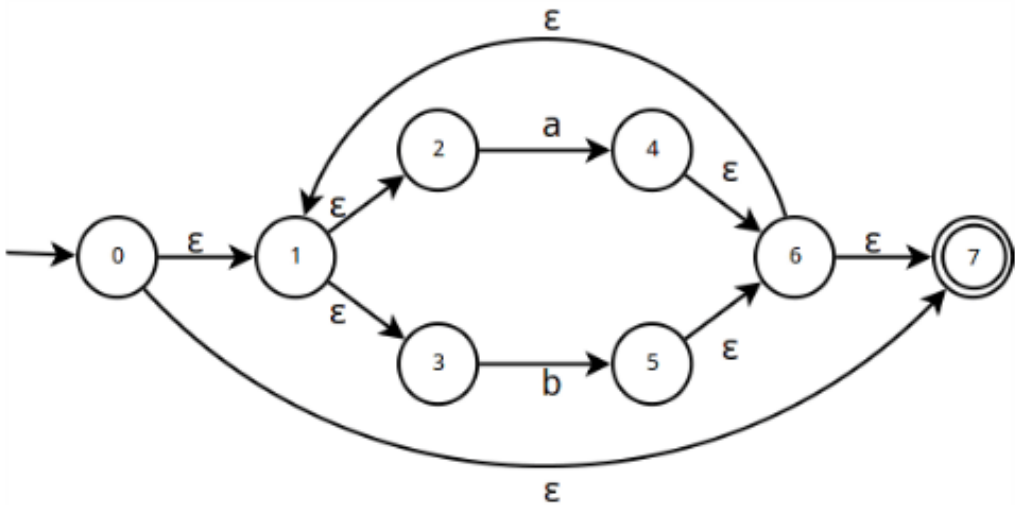
构造方法：汤普孙构造法、变式的构造法、展开法

展开法的结果和前两者不一样

Answer:

1.

NFA



Transition table

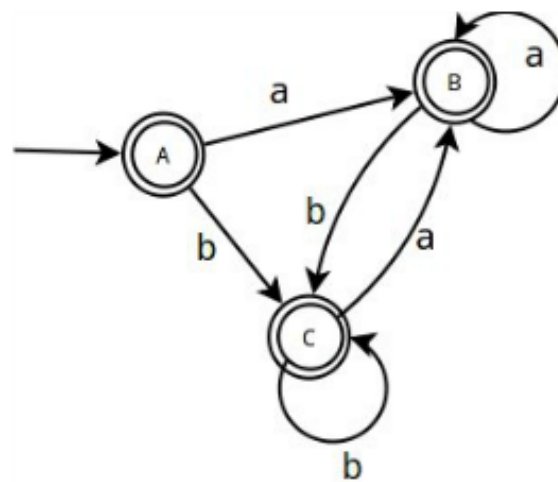
NFA State	DFA State	a	b
{0,1,2,3,7}	A	B	C
{1,2,3,4,6,7}	B	B	C
{1,2,3,5,6,7}	C	B	C

DFA

$r=s|t$

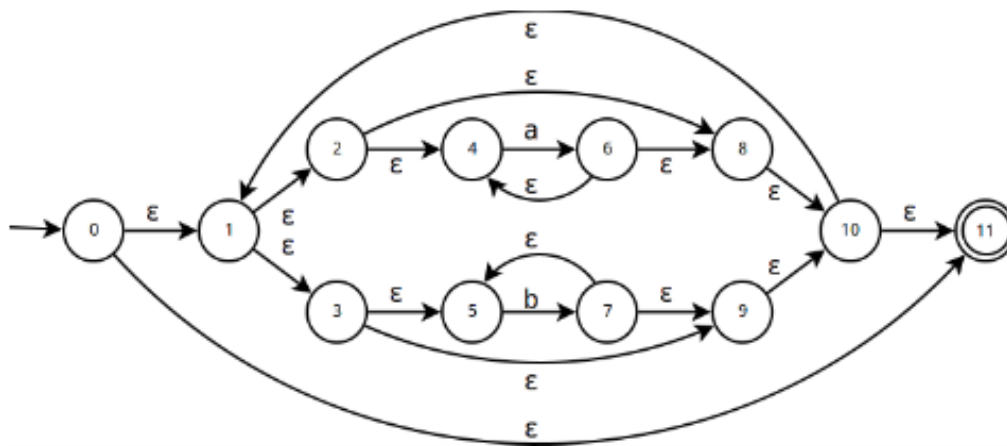
变式的构造法

$r=s^*$



2.

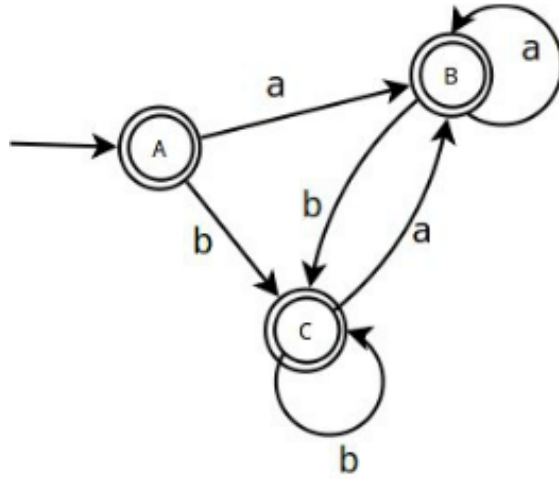
NFA



Transition table

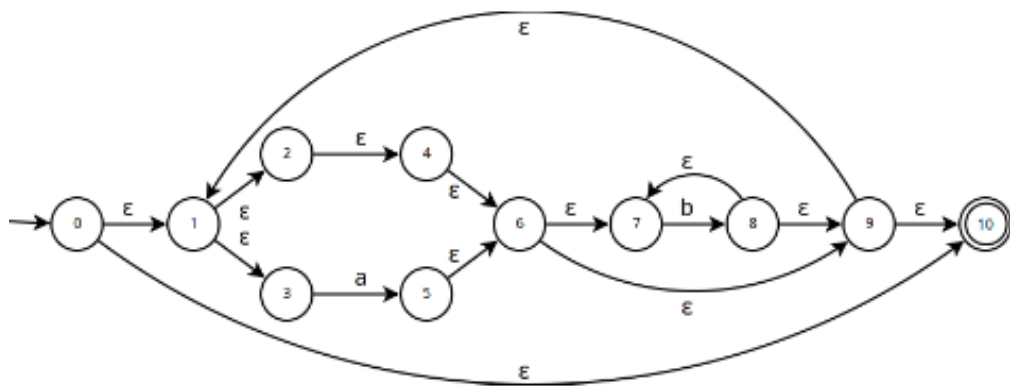
NFA State	DFA State	a	b
{0,1,2,3,4,5,8,9,10,11}	A	B	C
{1,2,3,4,5,6,8,9,10,11}	B	B	C
{1,2,3,4,5,7,8,9,10,11}	C	B	C

DFA



3.

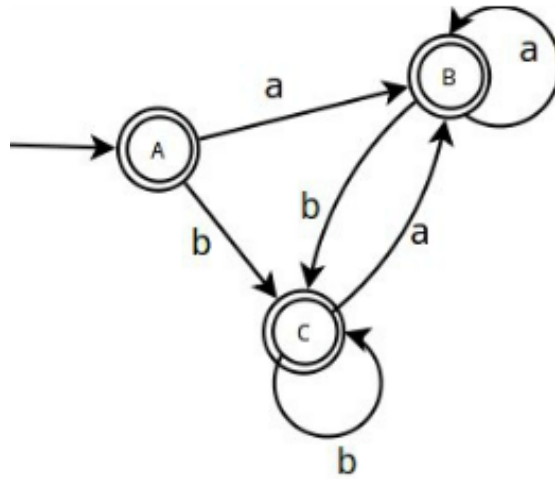
NFA



Transition table

NFA State	DFA State	a	b
{0,1,2,3,4,6,7,9,10}	A	B	C
{1,2,3,4,5,6,7,9,10}	B	B	C
{1,2,3,4,6,7,8,9,10}	C	B	C

DFA



附 1.3.4:

目标是描述 **最多包含一次** 相邻重复的字符串集合，即：

$$L = NR \cup OneRepeat$$

其中：

- NR 是无任何相邻重复的字符串。(第三问的解答)
- $OneRepeat$ 是恰好有一次相邻重复的字符串集合。

一个字符串中如果 **恰好有一次** 相邻重复，它的结构应为：

$$X \ xx \ Y$$

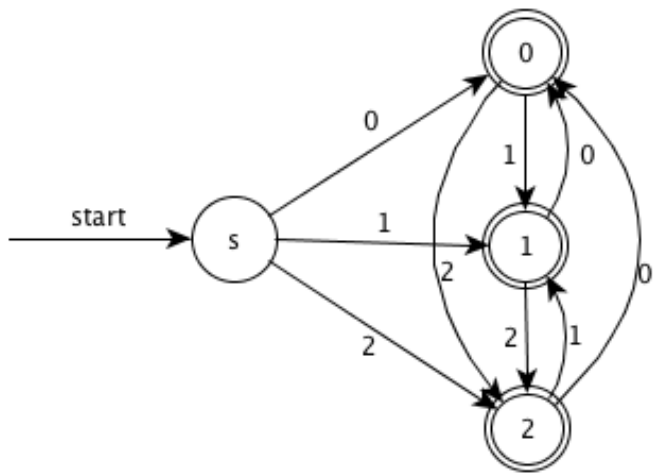
其中：

- X 是无相邻重复的前缀，且 **不能以 x 结尾**，否则会形成额外的相邻重复。
- xx 是唯一的相邻重复（唯一的 " xx " 位置）。
- Y 是无相邻重复的后缀，且 **不能以 x 开头**，否则 " xx " 后又出现 " x " 就会有第二次重复。

所以 **恰好有一次相邻重复** 的形式为：

$$OneRepeat = \bigcup_{x \in \{0,1,2\}} NR_{\text{end}}^x \ xx \ NR_{\text{start}}^x$$

解答第三问时你应该画了一个自动机:



举例而言,当我需要 X 且 x 为0时,上述自动机改为 仅有0一个接收态 ,其余布线不变,并可以通过自动机用相同的方式写出语言(Y 类似)

你无需算出最后的表达式,理解方法即可.

Assignment 2

2.1 (Read Through and understand Types and Programming Languages Chapter 5) Define a term for raising one number to the power of another.

2.2 Church encoding is a means of embedding data and operators into the λ calculus, the most familiar form being the Church numerals, a representation of the natural numbers using λ notation. Church numerals 0, 1, 2, ..., are defined as follows:

$$\begin{aligned} 0 &= \lambda s. \lambda z. z, \\ 1 &= \lambda s. \lambda z. s\ z, \\ 2 &= \lambda s. \lambda z. s(s\ z), \\ 3 &= \lambda s. \lambda z. s(s(s\ z)), \\ &\vdots \\ n &= \lambda s. \lambda z. s^n\ z, \end{aligned} \tag{1}$$

Church numerals takes two parameters s and z . Church numerals n means apply s to z n times. Note that church encoding cannot represent negative integers, we now try to encode all integer using untyped lambda calculus.

(a) Propose a method to extend church numerals to representation of integers. (Hint: you may try to use pairs). Give a concrete example for representation of integer -5 with your proposed method.

(b) Define a function *nat2int* that converts a natural number to your representation of correspondent integer.

(c) Based on this definition of integers, define the following arithmetic operations in lambda calculus:

(you can directly use operations on natural numbers defined before like add, multi, etc.)

(1) negation: neg n

(2) addition: addint $m\ n$

(3) multiplication: multint $m\ n$

2.3 Add appropriate type annotations to the following expressions, and state the type of the expression.

(a) $\lambda a. a + 4$

(b) $\lambda f. 3 + f()$

2.4 Infer the type of :

$$(\lambda x : \text{Int}. \lambda y : \text{Int}. x + y)\ 5 \tag{2}$$

Assignment 2

2.1 (Read Through and understand Types and Programming Languages Chapter 5) Define a term for raising one number to the power of another.

Answer:

$$pow = \lambda m. \lambda n. (n(times\ m))c_1$$

2.2 Church encoding is a means of embedding data and operators into the λ calculus, the most familiar form being the Church numerals, a representation of the natural numbers using λ notation. Church numerals 0, 1, 2, ..., are defined as follows:

$$\begin{aligned} 0 &= \lambda s. \lambda z. z, \\ 1 &= \lambda s. \lambda z. s\ z, \\ 2 &= \lambda s. \lambda z. s(s\ z), \\ 3 &= \lambda s. \lambda z. s(s(s\ z)), \\ &\vdots \\ n &= \lambda s. \lambda z. s^n\ z, \end{aligned} \tag{1}$$

Church numerals takes two parameters s and z . Church numerals n means apply s to z n times. Note that church encoding cannot represent negative integers, we now try to encode all integer using untyped lambda calculus.

(a) Propose a method to extend church numerals to representation of integers.(Hint: you may try to use pairs). Give a concrete example for representation of integer -5 with your proposed method.

(b) Define a function *nat2int* that converts a natural number to your representation of correspondent integer.

(c) Based on this definition of integers, define the following arithmetic operations in lambda calculus:

(you can directly use operations on natural numbers defined before like add, multi, etc.)

(1) negation: neg n

(2) addition: addint m n

(3) multiplication: multint m n

Answer:

(a) We can represent any integer n by a pair(a,b) and n is the difference between a and b . In other words, $n=a-b$. Since the integer value is more naturally represented if one of the pair is zero, we define the function *zro* to convert any pair to a pair include only one zero:

$$zro = \lambda z. iszro\ (fst\ z)\ z\ (iszro\ (snd\ z)\ z\ (zro\ (pair\ (prd\ (fst\ z))\ (prd\ (snd\ z))))$$

and we use fix-point combinator to implement recursion:

$$o = \lambda sz. \text{iszero} (fst\ z) \ z (\text{iszero} (snd\ z) \ z (s\ (pair\ (prd\ (fst\ z))\ (prd\ (snd\ z)))))$$

$$zro = fix\ o$$

In this representation:

理解zro函数可以实现二元组中含有0的形式

$$-5 = pair\ 0\ 5$$

(There are infinite pairs to encode -5 in this way, but we can always apply our function *zro* on it and get *pair* 0 5)

(b)

$$nat2int = \lambda z. pair\ z\ 0$$

(c)

(1)

$$neg = \lambda z. pair\ (snd\ z)\ (fst\ z)$$

(2)

$$addint = \lambda m. \lambda n. zro\ (pair\ (plus\ (fst\ m)\ (fst\ n))\ (plus\ (snd\ m)\ (snd\ n)))$$

(3)

$$multint = \lambda m. \lambda n. zro\ (pair\ (plus\ (times\ (fst\ m)\ (fst\ n))\ (times\ (snd\ m)\ (snd\ n)))\ (plus\ (times\ (fst\ m)\ (snd\ n))\ (times\ (snd\ m)\ (fst\ n))))$$

2.3 Add appropriate type annotations to the following expressions, and state the type of the expression.

(a) $\lambda a. a + 4$

(b) $\lambda f. 3 + f()$

Answer:

(a) With minimal annotations:

$$\lambda a : \mathbf{int}. a + 4$$

and the expression has type $\mathbf{int} \rightarrow \mathbf{int}$.

(b) With minimal annotations:

$$\lambda f : \mathbf{unit} \rightarrow \mathbf{int}. 3 + f()$$

and the expression has type $(\mathbf{unit} \rightarrow \mathbf{int}) \rightarrow \mathbf{int}$.

2.4 Infer the type of :

$$(\lambda x : \text{Int}. \lambda y : \text{Int}. x + y) 5 \quad (2)$$

Answer:

$$\frac{\frac{\frac{x : \text{Int}, y : \text{Int} \vdash x : \text{Int} \quad x : \text{Int}, y : \text{Int} \vdash y : \text{Int}}{x : \text{Int}, y : \text{Int} \vdash x + y : \text{Int}} \text{T-Var}}{x : \text{Int} \vdash \lambda y : \text{Int}. x + y : \text{Int} \rightarrow \text{Int}} \text{T-Abs}}{\vdash \lambda x : \text{Int}. \lambda y : \text{Int}. x + y : \text{Int} \rightarrow (\text{Int} \rightarrow \text{Int})} \text{T-Abs} \quad \vdash 5 : \text{Int} \text{T-App}$$
$$\frac{\vdash \lambda x : \text{Int}. \lambda y : \text{Int}. x + y : \text{Int} \rightarrow (\text{Int} \rightarrow \text{Int}) \quad \vdash 5 : \text{Int}}{\vdash (\lambda x : \text{Int}. \lambda y : \text{Int}. x + y) 5 : \text{Int} \rightarrow \text{Int}}$$

Assignment 3

March 2025

1. Prove the following expressions with the subtyping rules in slide Lecture 10.

$$(I) \{x : \{a : \text{Nat}, b : \text{Nat}\}, y : \text{Bool}\} <: \{x : \{a : \text{Nat}\}\}$$

$$(II) \{x : \{a : \text{Nat}, b : \{c : \text{Nat}, d : \text{Nat}\}\}, y : \text{Bool}\} <: \{x : \{a : \text{Nat}, b : \{c : \text{Nat}\}\}\}$$

$$(III) \{a : \{x : \{y : \text{Nat}, z : \text{Nat}\}, w : \text{Bool}\}, b : \text{Bool}\} <: \{a : \{x : \{y : \text{Nat}\}\}\}$$

$$(IV) (\{a : \text{Nat}\} \rightarrow \{b : \text{Nat}, c : \text{Nat}\}) <: (\{a : \text{Nat}\} \rightarrow \{b : \text{Nat}\})$$

2. Which of the following are subtypes of each other? Find such pairs and provide a proof for each one of them. For example, (II) is a subtype of (I). Find other pairs like this.

$$(I) \{\text{dogs} : \text{int}, \text{cats} : \text{int}\} \rightarrow \{\text{apples} : \text{int}\}$$

$$(II) \{\text{dogs} : \text{int}\} \rightarrow \{\text{apples} : \text{int}\}$$

$$(III) \{\text{dogs} : \text{int}\} \rightarrow \{\text{apples} : \text{int}, \text{kiwis} : \text{int}\}$$

$$(IV) \{\text{dogs} : \text{int}, \text{cats} : \text{int}, \text{mice} : \text{int}\} \rightarrow \{\text{apples} : \text{int}, \text{kiwis} : \text{int}\}$$

3. One presentation of labeled binary trees defines a tree to be either a leaf (with no label) or else an interior node with a numeric label and two child trees. Define a type `NatTree` and suitable operations for constructing, destructing, and testing trees. Write a function that performs a depth-first traversal of a tree and returns a list of the labels it finds.

4. Define a stream that yields successive elements of the Fibonacci sequence (1, 1, 2, 3, 5, 8, 13, . . .).

5. Write down principal unifiers (when they exist) for the following sets of constraints:

$$\{X = \text{Nat}, Y = X \rightarrow X\} \quad \{\text{Nat} \rightarrow \text{Nat} = X \rightarrow Y\}$$

$$\{X \rightarrow Y = Y \rightarrow Z, Z = U \rightarrow W\} \quad \{\text{Nat} = \text{Nat} \rightarrow Y\}$$

$$\{Y = \text{Nat} \rightarrow Y\} \quad \{\} \text{ (the empty set of constraints)}$$

6. Show that the type

$$\text{PairNat} = \forall X. (\text{CNat} \rightarrow \text{CNat} \rightarrow X) \rightarrow X;$$

can be used to represent pairs of numbers, by writing functions

```
pairNat : CNat → CNat → PairNat;
```

```
fstNat : PairNat → CNat;
```

```
sndNat : PairNat → CNat;
```

for constructing elements of this type from pairs of numbers and for accessing their first and second components.

Solutions for Assignment 3

1. SOLUTION:

(I)

$$\frac{\frac{}{\{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\text{Bool}\} <: \{x:\{a:\text{Nat}, b:\text{Nat}\}\}} \text{S-Rcdwidth} \quad \frac{\frac{}{\{a:\text{Nat}, b:\text{Nat}\} <: \{a:\text{Nat}\}} \text{S-Rcdwidth} \quad \frac{}{\{x:\{a:\text{Nat}, b:\text{Nat}\}\} <: \{x:\{a:\text{Nat}\}\}} \text{S-Rcddepth}}{} \text{S-Trans} \quad \{x:\{a:\text{Nat}, b:\text{Nat}\}, y:\text{Bool}\} <: \{x:\{a:\text{Nat}\}\}$$

(II)

$$\frac{\frac{}{\{x:\{a:\text{Nat}, b:\{c:\text{Nat}, d:\text{Nat}\}\}, y:\text{Bool}\} <: \{x:\{a:\text{Nat}, b:\{c:\text{Nat}, d:\text{Nat}\}\}\}} \text{S-Rcdwidth} \quad \frac{\frac{}{\{a:\text{Nat}\} <: \{a:\text{Nat}\}} \text{S-Refl} \quad \frac{\frac{}{\{c:\text{Nat}, d:\text{Nat}\} <: \{c:\text{Nat}\}} \text{S-Rcdwidth} \quad \frac{}{\{b:\{c:\text{Nat}, d:\text{Nat}\}\} <: \{b:\{c:\text{Nat}\}\}} \text{S-Rcddepth}}{} \text{S-Rcddepth} \quad \{x:\{a:\text{Nat}, b:\{c:\text{Nat}, d:\text{Nat}\}\} <: \{x:\{a:\text{Nat}, b:\{c:\text{Nat}\}\}\}} \text{S-Trans} \quad \{x:\{a:\text{Nat}, b:\{c:\text{Nat}, d:\text{Nat}\}\}, y:\text{Bool}\} <: \{x:\{a:\text{Nat}, b:\{c:\text{Nat}\}\}\}$$

(III)

$$\frac{\frac{}{\{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}, w:\text{Bool}\}, b:\text{Bool}\} <: \{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}, w:\text{Bool}\}\}} \text{S-Rcdwidth} \quad \frac{\frac{}{\{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}, w:\text{Bool}\}\} <: \{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}\}\}} \text{S-Rcdwidth} \quad \frac{\frac{}{\{y:\text{Nat}, z:\text{Nat}\} <: \{y:\text{Nat}\}} \text{S-Rcdwidth} \quad \frac{}{\{x:\{y:\text{Nat}, z:\text{Nat}\}\} <: \{x:\{y:\text{Nat}\}\}} \text{S-Rcddepth}}{} \text{S-Rcddepth} \quad \{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}, w:\text{Bool}\}, b:\text{Bool}\} <: \{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}\}\} \text{S-Trans} \quad \frac{}{\{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}\}\} <: \{a:\{x:\{y:\text{Nat}\}\}\}} \text{S-Trans} \quad \{a:\{x:\{y:\text{Nat}, z:\text{Nat}\}, w:\text{Bool}\}, b:\text{Bool}\} <: \{a:\{x:\{y:\text{Nat}\}\}\}$$

(IV)

$$\frac{\frac{}{\{a:\text{Nat}\} <: \{a:\text{Nat}\}} \text{S-Refl} \quad \frac{}{\{b:\text{Nat}, c:\text{Nat}\} <: \{b:\text{Nat}\}} \text{S-Rcdwidth}}{} \text{S-Arrow} \quad (\{a:\text{Nat}\} \rightarrow \{b:\text{Nat}, c:\text{Nat}\}) <: (\{a:\text{Nat}\} \rightarrow \{b:\text{Nat}\})$$

2. SOLUTION:

- (b) is a subtype of (a)
- (c) is a subtype of (b)
- (c) is a subtype of (d)
- (c) is a subtype of (a)

The proof is omitted.

3. SOLUTION:

```
Tree =  $\mu$ X. <leaf:Unit, node:{Nat,X,X}>;
leaf = <leaf=unit> as Tree;

► leaf : Tree

node =  $\lambda$ n:Nat.  $\lambda$ t1:Tree.  $\lambda$ t2:Tree. <node={n,t1,t2}> as Tree;

► node : Nat → Tree → Tree → Tree

isleaf =  $\lambda$ l:Tree. case l of <leaf=u> ⇒ true | <node=p> ⇒ false;

► isleaf : Tree → Bool

label =  $\lambda$ l:Tree. case l of <leaf=u> ⇒ 0 | <node=p> ⇒ p.1;

► label : Tree → Nat

left =  $\lambda$ l:Tree. case l of <leaf=u> ⇒ leaf | <node=p> ⇒ p.2;

► left : Tree → Tree

right =  $\lambda$ l:Tree. case l of <leaf=u> ⇒ leaf | <node=p> ⇒ p.3;

► right : Tree → Tree

append = fix ( $\lambda$ f:NatList→NatList→NatList.
                $\lambda$ l1:NatList.  $\lambda$ l2:NatList.
               if isnil l1 then l2 else
               cons (hd l1) (f (tl l1) l2));

► append : NatList → NatList → NatList

preorder = fix ( $\lambda$ f:Tree→NatList.  $\lambda$ t:Tree.
                if isleaf t then nil else
                cons (label t)
                    (append (f (left t)) (f (right t))));

► preorder : Tree → NatList

t1 = node 1 leaf leaf;
t2 = node 2 leaf leaf;
t3 = node 3 t1 t2;
t4 = node 4 t3 t3;
l = preorder t4;
hd l;

► 4 : Nat
hd (tl l);

► 3 : Nat
hd (tl (tl l));

► 1 : Nat
```

3. 标记二叉树的一种表示方式将树定义为叶子（没有标签）或具有数字标签和两个子树的内部节点。定义一个类型NatTree以及用于构建、销毁和测试树的适当操作。编写一个函数，执行树的深度优先遍历，并返回它找到的标签列表。

4. SOLUTION:

```
fib = fix (λf: Nat→Nat→Stream. λm:Nat. λn:Nat. λ_:Unit.
          {n, f n (plus m n)}) 0 1;
```

► fib : Stream

5. SOLUTION:

$\{X = \text{Nat}, Y = X \rightarrow X\}$	$[X \mapsto \text{Nat}, Y \mapsto \text{Nat} \rightarrow \text{Nat}]$
$\{\text{Nat} \rightarrow \text{Nat} = X \rightarrow Y\}$	$[X \mapsto \text{Nat}, Y \mapsto \text{Nat}]$
$\{X \rightarrow Y = Y \rightarrow Z, Z = U \rightarrow W\}$	$[X \mapsto U \rightarrow W, Y \mapsto U \rightarrow W, Z \mapsto U \rightarrow W]$
$\{\text{Nat} = \text{Nat} \rightarrow Y\}$	Not unifiable
$\{Y = \text{Nat} \rightarrow Y\}$	Not unifiable
$\{\}$	$[\]$

6. SOLUTION:

```
pairNat = λn1:CNat. λn2:CNat.
          λX. λf:CNat→CNat→X. f n1 n2;
fstNat = λp:PairNat. p [CNat] (λn1:CNat. λn2:CNat. n1);
sndNat = λp:PairNat. p [CNat] (λn1:CNat. λn2:CNat. n2);
```

HW4

1. Given the grammar $G(E)$ of the Lisp language as follows:

$$E \rightarrow L \mid a$$

$$L \rightarrow (S)$$

$$S \rightarrow SS \mid E$$

1. Write a **leftmost derivation** for the sentence `"(a(aa))"`.
2. **Eliminate left recursion** in the grammar $G(E)$.
3. Compute the **First** and **Follow** sets for all non-terminal symbols after eliminating left recursion.
4. Construct an **LL(1) parsing table** for the grammar after eliminating left recursion, and use it to show that the original grammar $G(E)$ is **not LL(1)**.
5. Using your parsing table, show a **correct parsing process** for the sentence `"(aa)"`.

2. Let the augmented grammar $G'(E')$ be defined as follows:

$$(0) \quad E' \rightarrow E$$

$$(1) \quad E \rightarrow L$$

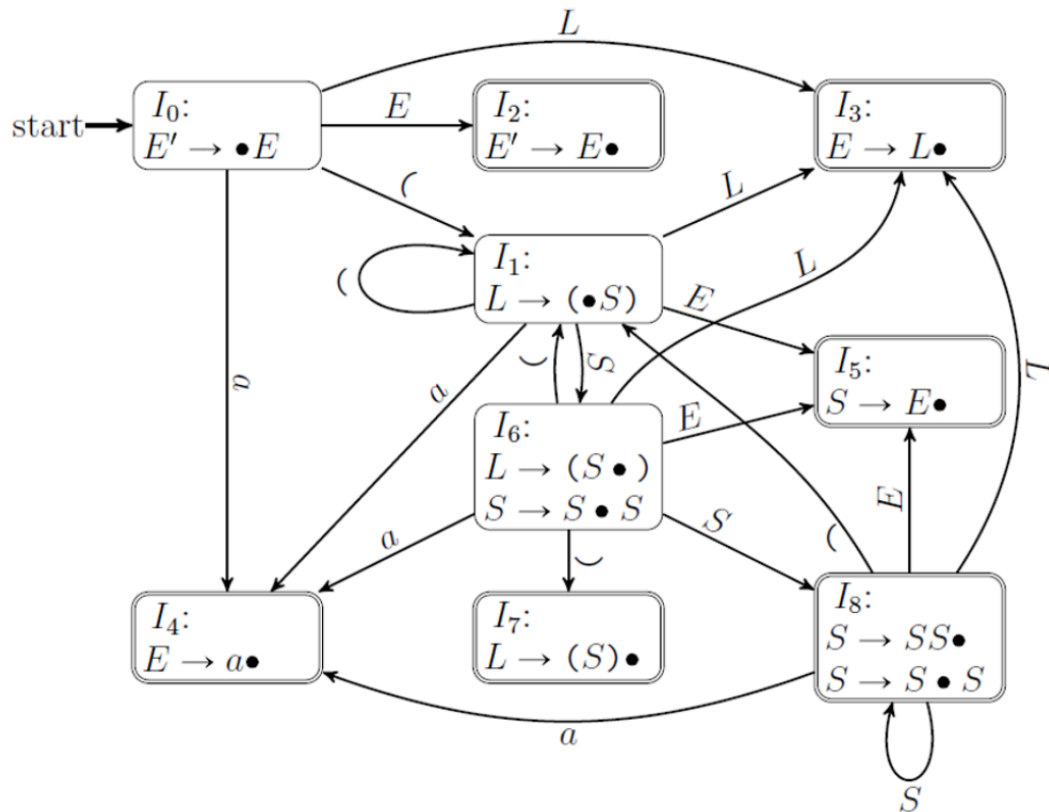
$$(2) \quad E \rightarrow a$$

$$(3) \quad L \rightarrow (S)$$

$$(4) \quad S \rightarrow SS$$

$$(5) \quad S \rightarrow E$$

The LR(0) item DFA (deterministic finite automaton) for grammar $G'(E')$ is shown in the figure (only core items are listed for each state):



1. Find the complete LR(0) item set corresponding to state I_1 .
2. Write the regular expressions corresponding to viable prefixes composed entirely of terminal symbols.
3. Construct the **SLR parsing table** for this grammar, and assume that the **concatenation operation (i.e., SS) is right-associative**.
4. Use your parsing table to analyze the input string `"(aa)"` and provide the parsing process.

HW4_sol

▼ 1Answer:

(1) A leftmost derivation for the sentence "a(aa)":

$$\begin{aligned}
 E &\Rightarrow_{lm} L \\
 &\Rightarrow_{lm} (S) \\
 &\Rightarrow_{lm} (SS) \\
 &\Rightarrow_{lm} (ES) \\
 &\Rightarrow_{lm} (aS) \\
 &\Rightarrow_{lm} (aE) \\
 &\Rightarrow_{lm} (aL) \\
 &\Rightarrow_{lm} (a(S)) \\
 &\Rightarrow_{lm} (a(SS)) \\
 &\Rightarrow_{lm} (a(ES)) \\
 &\Rightarrow_{lm} (a(aS)) \\
 &\Rightarrow_{lm} (a(aE)) \\
 &\Rightarrow_{lm} (a(aa))
 \end{aligned}$$

(2) Grammar after eliminating left recursion:

$$\begin{aligned}
 E &\rightarrow L \mid a \\
 L &\rightarrow (S) \\
 S &\rightarrow ES' \\
 S' &\rightarrow SS' \mid \varepsilon
 \end{aligned}$$

(3) First and Follow sets of non-terminal symbols after removing left recursion:

$$\begin{aligned}
 \text{First}(E) &= \text{First}(S) = \{a, (\} \\
 \text{First}(L) &= \{(\} \\
 \text{First}(S') &= \{a, (, \varepsilon\} \\
 \text{Follow}(E) &= \text{Follow}(L) = \{a, (,), \$\} \\
 \text{Follow}(S) &= \text{Follow}(S') = \{a, (,)\}
 \end{aligned}$$

(4) LL(1) Parsing Table:

	a	$($	$)$	$\$$
E	$E \rightarrow a$	$E \rightarrow L$		
L		$L \rightarrow (S)$		
S	$S \rightarrow ES'$	$S \rightarrow ES'$		
S'	$S' \rightarrow SS' \mid \varepsilon$	$S' \rightarrow SS' \mid \varepsilon$	$S' \rightarrow \varepsilon$	

(5) Parsing process for the sentence "aa":

Input String	Parse Stack	Parsing Action
(aa)\$	E\$	$E \rightarrow L$
(aa)\$	L\$	$L \rightarrow (S)$
(aa)\$	(S)\$	match-advance
aa)\$	S)\$	$S \rightarrow ES'$

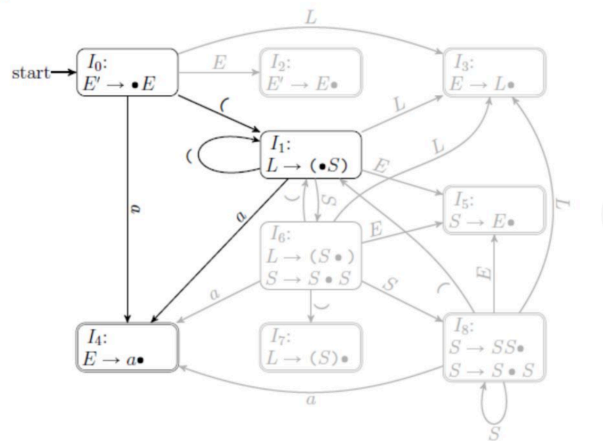
aa)\$	ES')\$	$E \rightarrow a$
aa)\$	aS')\$	match-advance
a)\$	S')\$	$S' \rightarrow SS'$
a)\$	ES'S')\$	$S \rightarrow ES'$
a)\$	ES'S')\$	$E \rightarrow aS'$
)\$	S'S')\$	$S' \rightarrow \varepsilon$
)\$	S')\$	$S' \rightarrow \varepsilon$
)\$	)\$	match-advance
\$	\$	Parsing successful

▼ 2Answer:

(1)

$$\begin{aligned} & \overline{\{L \rightarrow (\bullet S)\}} \\ &= \{L \rightarrow (\bullet S), S \rightarrow \bullet S S, S \rightarrow \bullet E, E \rightarrow \bullet L, \\ & \quad E \rightarrow \bullet a, L \rightarrow \bullet (S)\} \end{aligned}$$

(2)



$$(* (a \mid \varepsilon))$$

(3) SLR Parsing Table:

First sets:

$$\begin{aligned} \text{First}(E) &= \text{First}(S) = \{a, (\} \\ \text{First}(L) &= \{(\} \end{aligned}$$

Follow sets:

$$\begin{aligned} \text{Follow}(E) &= \text{Follow}(L) = \{a, (,), \$\} \\ \text{Follow}(S) &= \{a, (,)\} \end{aligned}$$

SLR Parsing Table:

		action				goto	
--	--	--------	--	--	--	------	--

State	a	(	)	\$	E	L	S
0	s4	s1			2	3	
1	s4	s1			5	3	6
2				acc			
3	r1	r1	r1	r1			
4	r2	r2	r2	r2			
5	r5	r5	r5				
6	s4	s1	s7		5	3	8
7	r3	r3	r3	r3			
8	s4	s1	r4		5	3	8

(4) Parsing process for the input string "(aa)":

Remaining Input	Parsing Stack	Action
(aa)\$	0	shift
aa)\$	0(1	shift
a)\$	0(1a4	$reduce E \rightarrow a$
a)\$	0(1E5	$reduce S \rightarrow E$
a)\$	0(1S6	shift
)\$	0(1S6a4	$reduce E \rightarrow a$
)\$	0(1S6E5	$reduce S \rightarrow E$
)\$	0(1S6S8	$reduce S \rightarrow SS$
)\$	0(1S6	shift
\$	0(1S6)7	$reduce L \rightarrow (S)$
\$	0L3	$reduce E \rightarrow L$
\$	0E2	accept